

HAETAETAE-2 구현 검증의 대수적 지도

순수 대수학자와 대수기하학자를 위한 동반 안내서

HAETAETAE top-down EasyCrypt 작업 문서

상태 기준: 2026년 8월 10일 (Week 13 종료)

Abstract

HAETAETAE-2는 $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ 위의 모듈 격자 서명(module-lattice signature)이다. 그러나 이 저장소에서 실제로 증명하는 대상은 환(ring) 자체에 머물지 않는다. 논문의 원소(element)와 모듈 사상(module map)을 정준 바이트열(canonical byte string)로 표현하고, 그 바이트열을 유한 메모리(finite memory)에 배치한 뒤, 실제 생성된 Jasmin 절차가 그 표현을 보존하거나 역산한다는 사실까지 내려간다. 이 문서는 그 과정을 “대수적 대상(algebraic object)–표현 사상(representation map)–상태기계(state machine) refinement”의 세 층으로 재구성한다.

최신 감사 기준선인 스냅샷 시점에는 67개 authored EasyCrypt 타깃이 manifest에 들어간 aggregate -no-eco fresh compile을 통과했다. Week 10의 rANS encoder 타깃 6개에 이어 Week 11의 encoder closure 타깃 7개, Week 12의 decoder semantic-refinement 타깃 8개, Week 13의 actual core composition 타깃 2개가 추가되었다. 닫힌 핵심은 비밀키의 992 바이트 검증기 접두부, raw/internal Sign-Verify의 32바이트 μ 일치, challenge suffix 흡수, 서명 prefix round trip, HBZ 계수(coefficient)–기호(symbol) leaf 역함수(inverse function), 실제 13-symbol rANS 표의 certificate, 순수 rANS 단계와 byte-stack 정리(theorem), 실제 suffix copy, encoder–decoder harness의 제어, 그리고 actual encoder 성공 출력 전체와 순수 TraceBytes(SymbolList(symbols₀)) 사이의 정확한 등식(equality), exact trace에서 1024개 기호(symbol)를 복원하는 actual decoder 정제(refinement), 그리고 두 정리(theorem)와 actual suffix copy를 Mode2RansActualHarness.run에서 합성한 core 역함수(inverse function)다. 마지막 정리(theorem)는 encoder 성공을 전제로 요구하지 않고, 실패 시 decoder를 실행하지 않는 가지(branch)와 성공 시 1024개 기호(symbol)를 복원하는 가지를 함께 보이는 성공 조건부 부분정확성(success-conditioned partial correctness)이다. 반면 실제 encoder 성공 증인(success witness), encoder/decoder 종료성(termination), production full-HBZ wrapper, 전체 suffix codec, 완료된 API trace의 저장 μ 를 연결하는 product/replay 운송, 전체 기능정확성과 구현 수준 보안은 열려 있다. 따라서 이 문서는 완성 증명을 선언하지 않고, 정확히 어디까지 commuting diagram이 닫혔는지를 표시한다.

한 문장 요약

현재 성과는 환(ring) 위의 전체 서명 스킴(signature scheme)을 이미 검증했다는 뜻이 아니라, 그 스킴의 몇몇 결정론적 표현 사각형(representation square)이 실제 생성 코드까지 가환(commute)함을 증명했고, 다음 핵심 gate가 actual_encode_hb_z1_full과 decode_hb_z1_full을 prepare/apply 역함수(inverse function) 및 증명된 rANS core와 합성하는 성공 조건부 full-HBZ wrapper 정리(theorem) 임을 뜻한다. GO-CORE는 encoder와 decoder의 개별 간선(edge)에 이어 actual suffix copy를 통한 core 가환 사각형(commutative square)까지 닫혔다는 판정이다. production full-HBZ wrapper나 전체 codec이 완료됐다는 판정은 아니다.

목차

1 독자 계약과 전체 지도

1.1	이 문서가 가정하는 것과 설명하는 것	4
1.2	상태어는 논리적 양화사(quantifier)를 포함한다	4
1.3	대수기하학은 어디까지 관련되는가	5
1.4	먼저 기억할 네 문장	5
2	대수적 대상(algebraic object): 몫환(quotient ring)에서 서명 바이트까지	6
2.1	기본 환(ring)과 mode 2 매개변수	6
2.2	KeyGen을 모듈 사상(module map)으로 보기	6
2.3	Sign과 Verify가 만나는 transcript	7
2.4	서명 표현은 정준 부분집합(canonical subset) 위의 부분동형(partial isomorphism)이다	7
2.5	어떤 대수적 사실이 어느 층에 있는가	8
3	표현 사상(representation map)과 프로그램 논리(program logic)의 번역 사전	8
3.1	배열과 메모리는 서로 다른 대상이다	8
3.2	주소의 두 표현과 정준 범위(canonical range)	9
3.3	절차는 전역 함수(total function)가 아니라 부분적 상태전이(partial state transition)다	9
3.4	정제(refinement)는 가환사각형(commutative square)이다	10
3.5	프레임(frame)은 쓰기 지지집합(write support)에 대한 정리(theorem)다	10
3.6	정준성(canonicity)은 대표자(representative) 선택이다	10
3.7	exact trace와 ghost observation	11
3.8	“zero-loss”의 정확한 범위	11
3.9	EasyCrypt 정리(theorem) 하나를 수학 문장으로 읽는 예	11
4	현재 닫힌 증명 사슬	12
4.1	두 개의 주 진행선	12
4.2	1단계: packed secret key의 검증키 몫(quotient)	12
4.3	2단계: 로컬 배열에서 public memory까지	13
4.4	3단계: Sign과 Verify의 generated raw μ	13
4.5	4단계: 전체 메모리 등식(whole-memory equality)에서 read-region 등식(read-region equality)으로	14
4.6	5단계: position 64 challenge suffix	14
4.7	6단계: 실제 caller의 control과 accepted Verify	14
4.8	7단계: 실제 signature prefix의 section-retraction	15
4.9	8단계: HBZ coefficient-symbol leaf	15
4.10	9단계: 실제 표와 순수 rANS 수학	16
4.11	10단계: actual encoder의 국소 의미론(local semantics)	16
4.12	11단계: actual encoder의 전역 trace closure	17
4.13	12단계: actual decoder의 exact-trace 의미론(semantics)	17
4.14	13단계: actual encoder-copy-decoder core 역함수(inverse function)	18
4.15	닫힌 사슬의 정확한 끝점	19
5	현재 전선: HBZ/rANS를 유한 동역학계(finite dynamical system)로 읽기	20
5.1	왜 production full-HBZ wrapper가 지금의 단일 gate인가	20
5.2	13-symbol 분할(partition)	20
5.3	순수 단계는 유클리드 나눗셈(Euclidean division)의 역함수(inverse function)다	21

5.4	정규화 바이트(normalization byte)는 잃어버린 몫(quotient)의 좌표(coordinate)다	21
5.5	전체 symbol list의 순수 trace	22
5.6	Week 10에서 감사된 여섯 encoder 타깃	22
5.7	Week 11에서 닫힌 actual encoder 불변식(invariant)	23
5.8	Week 12에서 닫힌 actual decoder 불변식(invariant)	23
5.9	Week 13에서 닫힌 actual harness core 합성(composition)	24
5.10	full HBZ wrapper의 올바른 목표 모양	25
6	열린 경계와 과장 금지선	27
6.1	가장 좁지만 중요한 의미론(semantics) 간극(gap): product/replay	27
6.2	accepted-path와 legitimate-signature correctness는 다른 방향이다	27
6.3	full signature codec에서 남은 표현론	28
6.4	대수(algebra) · 수치(numerics) · 확률(probability) 층(layer)에 남은 의무	28
6.5	최상위 기능정확성은 아직 명세다	29
6.6	보안 합성식은 아직 조건부 인터페이스다	29
6.7	신뢰 경계와 범위 밖 항목	29
6.8	현재 주장 행렬	30
7	의존성에 따른 다음 작업과 참여 지점	31
7.1	현재 단일 우선순위	31
7.2	1단계 완료: encoder	31
7.3	2단계 완료: decoder	32
7.4	3단계 완료: actual harness inverse	33
7.5	4단계 현재 완료 기준: production full-HBZ wrapper	33
7.6	5-6단계: 작은 부모부터 닫기	33
7.7	동결된 product/replay 선은 별도로 다룬다	34
7.8	순수 대수학자가 바로 기여할 수 있는 문제	34
7.9	대수기하학자의 기술이 유용한 곳	35
7.10	새 정리(theorem)를 제출할 때의 판정 체크리스트	35
A	정리(theorem)와 소스 인덱스	35
A.1	상태와 범위 문서	35
A.2	packed key와 memory bridge	36
A.3	μ , challenge, actual API trace	37
A.4	signature prefix와 HBZ leaf	38
A.5	rANS table, 순수 trace, actual boundary	38
A.6	최상위 목표와 가정 표면	41
B	기호와 용어 사전	41
C	재현과 독해 경로	43
C.1	이 안내서만 빌드하기	43
C.2	전체 증명 파이프라인	43
C.3	30분 독해 경로	44
C.4	증명 참여 전 만나질 경로	44

1 독자 계약과 전체 지도

1.1 이 문서가 가정하는 것과 설명하는 것

이 문서는 독자가 환(ring), 몫환(quotient ring), 자유 모듈(free module), 동치관계(equivalence relation)와 몫집합(quotient set), 가환도식(commutative diagram), 귀납법(induction)에 익숙하다고 가정한다. 수학 용어는 이처럼 문서의 첫 등장과 표제·도표·용어 사전에서 한국어 뒤에 영어 원어를 병기한다. 같은 문맥에서 반복할 때에는 가독성을 위해 한국어만 쓸 수 있다. 반대로 다음 지식은 가정하지 않는다.

- HAETAE 또는 모듈 격자 서명(module-lattice signature)의 설계;
- Jasmin의 문법과 메모리 모델;
- EasyCrypt의 Hoare logic, pRHL, 전술 언어;
- rANS entropy coding의 구현 관례.

목표는 모든 소스 코드를 줄마다 설명하는 것이 아니다. 각 증명 의무에 대해 “대상(object)은 무엇인가”, “어떤 사상(map)이 가환(commute)해야 하는가”, “정의역(domain)을 제한하는 전제(premise)는 무엇인가”, “현재 어느 화살표(arrow)까지 실제 절차에 대해 증명되었는가”를 답하는 것이 목표다. 프로젝트의 범위는 mode 2와 고정된 명세·구현 스냅샷에 한정된다 [7].

번역 원리 1.1 (세 층 번역). 저장소의 정리(theorem)는 다음 세 층을 오가는 명제(proposition)로 읽는다.

1. **대수 층(algebraic layer)**: R_q -모듈(module) 원소, 분해(decomposition), challenge, 논문 알고리즘;
2. **표현 층(representation layer)**: 정준 바이트열(canonical byte string), 접두부 투영(prefix projection), 메모리 구간(memory region), 고정폭 정수(fixed-width integer);
3. **실행 층(execution layer)**: 실제 생성 절차, 분기, 루프, 반환값과 trace 관측값(observation).

한 층에서 성립하는 등식(equality)을 다른 층으로 올리거나 내리려면 반드시 명명된 representation/refinement 정리(theorem)가 필요하다. 같은 수학적 의도를 구현한다는 사실만으로 도식(diagram)이 자동으로 가환(commute)하지 않는다.

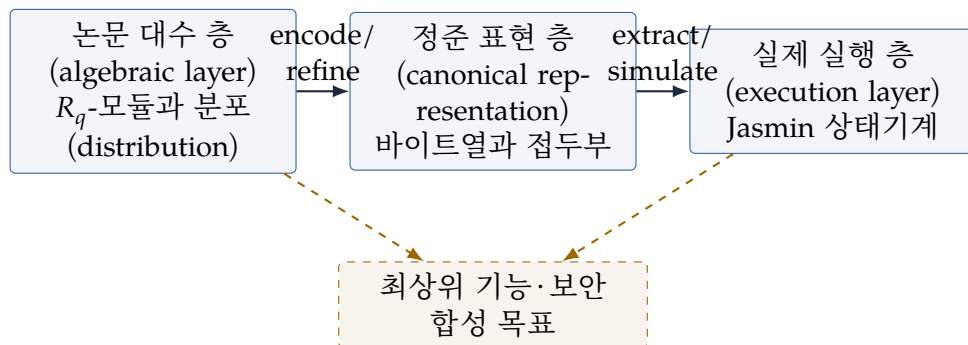


그림 1: 전체 증명 구조. 수평 화살표의 일부만 단혔고, 최상위 합성 화살표는 아직 부분적이다.

1.2 상태어는 논리적 양화사(quantifier)를 포함한다

운영상 source of truth는 CLAIM_LEDGER.md이며 [3], 의존성은 THEOREM_GRAPH.md에 기록된다 [8]. 이 문서에서 상태어는 다음 의미로만 사용한다.

표시	정확한 뜻	자동으로 따라오지 않는 것
PROVED	표시된 전제 아래 명명된 EasyCrypt 선언이 fresh-compile된다.	부모 주장, 종료성(termination), losslessness, 분포 동일성(distributional equality)
PARTIAL	자식 명제(proposition) 또는 제한된 방향은 증명되었지만 명명된 부모가 열려 있다.	제한을 제거한 전역 명제(global statement)
SPECIFIED	목표를 나타내는 연산(operation) · 술어(predicate) · 인터페이스가 컴파일된다.	그 목표의 증명
BLOCKED	구체적인 extraction, 의미론(semantics), 수치 · 확률 정리(theorem)가 없어 합성할 수 없다.	난점이 해결 불가능하다는 주장
DEFERRED	시간 제한 뒤 명시적인 논문 경계로 동결했다.	거짓 또는 불필요하다는 주장

특히 “Hoare partial correctness가 증명되었다”는 문장은

절차가 반환한다면 사후조건이 성립한다

는 뜻이다. 절차가 모든 입력에서 반환한다거나, 확률적 재시도가 거의 확실히 끝난다는 뜻이 아니다. 이 구분은 KeyGen과 Sign의 rejection loop 때문에 본질적이다.

1.3 대수기하학은 어디까지 관련되는가

R_q 는 물론 아핀 스킴(affine scheme) $\text{Spec } R_q$ 를 정의한다. 또한 메모리의 부분구간 제한(local restriction), 국소 등식(local equality), frame 조건은 제한사상(restriction map)과 접합(gluing)을 떠올리게 한다. 그러나 현재 EasyCrypt 파일은 스킴(scheme), 다형체(variety), 스펙트럼(spectrum), 층(sheaf), 코호몰로지(cohomology)를 형식화하지 않는다. 증명 대상은 주로 유한 집합(finite set) 위의 배열, 고정폭 word, 상태 전이(state transition), 정수 나눗셈(integer division)이다.

비유(형식화된 정리(theorem) 아님).

메모리 m 을 주소집합(address set) 위의 단면(section)처럼 보고, 구간 U 에서의 동일성을 $m_1 \upharpoonright_U = m_2 \upharpoonright_U$ 로 읽으면 region-local hash 정리(theorem)의 모양을 빠르게 이해할 수 있다. 하지만 이 비유에서 층 공리(sheaf axiom)나 스킴 구조(scheme structure)를 사용해 EasyCrypt 결론을 도출하지 않는다. 실제 근거는 점별 load8 등식과 절차의 read footprint다.

대수기하학자가 특히 주의할 점은 hash 함수다. SHAKE 상태전이는 일반적으로 R_q -선형사상(linear map)도 정칙사상(regular morphism)도 아니다. 이 문서에서 hash는 유한 바이트열을 읽는 결정론적 상태기계로 취급한다. 따라서 hash 입력의 동일성에서 출력 동일성(output equality)을 얻는 이유는 대수적 함수성(algebraic functionality)보다 실제 두 프로그램의 관계적 실행 정리(relational execution theorem)에 있다.

1.4 먼저 기억할 네 문장

1. 최상위 FC_KeyGen, FC_Sign, FC_Verify는 아직 목표 인터페이스이지 완료된 정리(theorem)가 아니다.

2. $\text{prefix}_{992}(sk) = vk$ 와 raw/internal μ -접두부 일치는 실제 생성 절차까지 증명되었지만, 전체 API 보안으로 자동 승격되지 않는다.
3. HBZ/rANS의 순수 역함수 구조, 실제 encoder/decoder의 개별 trace 정제(refinement), actual copy를 포함한 core harness 역함수(inverse function)는 닫혔지만, production full-HBZ wrapper 화살표는 열려 있다.
4. 최종 암호학적 가정과 구현 증명 의무를 분리한다. MLWE/BST-MSIS/ROM 인터페이스 외의 구현 정확성은 가정으로 숨기지 않는다.

이 네 문장은 이후 모든 장의 상태 판정 규칙이다.

2 대수적 대상(algebraic object): 몫환(quotient ring)에서 서명 바이트까지

2.1 기본 환(ring)과 mode 2 매개변수

고정된 명세에서

$$R = \mathbb{Z}[X]/(X^{256} + 1), \quad R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \quad q = 64513$$

이고, mode 2의 모듈 차원(module dimension)은 $(k, \ell) = (2, 4)$, 작은 비밀계수(secret coefficient) 범위는 $\eta = 1$, binary challenge의 Hamming weight는 $\tau = 58$ 이다 [1]. 각 R_q 원소는 차수 < 256 인 다항식(polynomial)의 동치류(equivalence class)이며, 계수벡터(coefficient vector)를 택하면 $\mathbb{Z}_q^{256}$ 와 집합(set)으로 식별할 수 있다.

명세는 $q \equiv 1 \pmod{512}$ 를 택해 $X^{256} + 1$ 이 $\mathbb{F}_q$ 에서 완전히 분해되고 NTT를 사용할 수 있게 한다. 따라서 종이 위에서는 중국인의 나머지 정리(Chinese remainder theorem)가 주는 동형사상(isomorphism)

$$R_q \simeq \prod_{i=1}^{256} \mathbb{F}_q$$

를 떠올릴 수 있다. 다만 현재 저장소의 최상위 구현 보안 인터페이스에는 `obl_public_key_algebra_ntt`가 별도 의무로 남아 있다. 즉 이 동형사상(isomorphism)의 존재와 실제 NTT 루틴의 정확성을 같은 주장으로 취급하면 안 된다.

주의 2.1 (수학 배경과 구현 정리(implementation theorem)의 분리). 이 절의 환(ring)·모듈(module) 서술은 고정된 HAETAE 명세를 설명한다. “실제 Jasmin NTT가 이 동형사상(isomorphism)을 정확히 계산한다”는 구현 정리(implementation theorem)는 별도의 명제(proposition)와 전제(premise)가 필요하다. 본 안내서의 **PROVED**표시는 오직 easycrypt/ 아래 명명된 정리(theorem)에 붙인다.

2.2 KeyGen을 모듈 사상(module map)으로 보기

논문의 KeyGen 데이터 흐름에서 seed로부터 $a_{\text{gen}} \in R_q^k$ 와 $A_{\text{gen}} \in R_q^{k \times (\ell-1)}$ 를 만들고, 작은 원소(element) $s_{\text{gen}} \in R^{\ell-1}$, $e_{\text{gen}} \in R^k$ 를 표본추출(sample)하여

$$b = a_{\text{gen}} + A_{\text{gen}} s_{\text{gen}} + e_{\text{gen}} \in R_q^k$$

를 계산한다. 그 뒤 계수별 분해(coefficientwise decomposition)

$$b = b_0 + 2b_1, \quad (b_0, b_1) = (\text{LowBits}_{vk}(b), \text{HighBits}_{vk}(b))$$

를 사용하고, 검증키는 $vk = (seedA, b_1)$ 로 압축된다. 비밀키의 packed representation은 Sign이 같은 검증키 성분을 다시 읽을 수 있도록 그 바이트 표현을 앞부분에 포함한다.

이 저장소에서 가장 먼저 닫힌 합성 seam은 전체 KeyGen 분포(distribution)가 아니라 바로 이 포함관계(inclusion relation)다. 바이트 배열의 집합(set)을 S , 검증키 배열의 집합을 V 라 하고

$$\pi_{992} : S \rightarrow \text{Byte}^{992}, \quad \pi_{992}(s) = s[0..992)$$

로 두면, 종료한 실제 mode 2 internal KeyGen 결과 (vk, sk) 에 대해

$$\pi_{992}(sk) = \pi_{992}(vk)$$

가 성립한다. 오른쪽 배열은 더 크지만 첫 992바이트만 비교한다.

이를 몫(quotient)으로 정확히 표현할 수도 있다. S 에

$$s \sim_{992} s' \iff s[0..992) = s'[0..992)$$

를 정의하면 π_{992} 는 $S/\sim_{992}$ 를 분류한다. 현재 정리(theorem)는 “packed secret key의 $\sim_{992}$ -동치류(equivalence class)가 public key의 표현과 같다”는 명제(proposition)다. 비밀키 전체의 등식(equality)이 아니다.

2.3 Sign과 Verify가 만나는 transcript

고정 명세에서 Sign과 Verify가 공통으로 재구성해야 하는 첫 관측값은 메시지 digest μ 다. 필요한 부분만 추리면

$$\mu = H_{\text{gen}}(seedA, b_1, M),$$

그리고 challenge 입력은

$$\rho = H(\text{HighBits}_h(w), \text{LSB}(\lfloor y_0 \rfloor), \mu)$$

의 모양을 갖는다 [1]. Sign은 packed secret key의 992바이트 접두부에서 $(seedA, b_1)$ 를 읽고, Verify는 public key 메모리에서 같은 바이트를 읽는다.

현재 raw/internal generated 경계에서 증명된 중심 등식은

$$\text{take}_{32}(\mu_{\text{Sign}}^{64}) = \mu_{\text{Verify}}^{32}$$

이다. 이어서 동일한 challenge 상태와 position 64에서 이 32바이트를 흡수하면 두 최종 상태와 position이 같다. 이 등식은 실제 generated hash/absorb 절차를 직접 포함하는 관계적 정리(relational theorem)다.

아직 증명되지 않은 것은 ρ 전체다. 특히 $\text{HighBits}_h(w)$ 와 LSB 표현의 실제 Sign-Verify 일치, 전체 challenge 호출, 분포·rejection 분석은 별도 의무다. 그러므로 닫힌 μ 사각형을 전체 challenge 또는 전체 $\delta_{\text{Sign}} = 0$ 로 확장할 수 없다.

2.4 서명 표현은 정준 부분집합(canonical subset) 위의 부분동형(partial isomorphism)이다

mode 2 서명은 1474바이트이고, 실제 추출된 layout은

$$\begin{array}{cccc} \underbrace{[0, 32)} & \parallel & \underbrace{[32, 1056)} & \parallel & \underbrace{[1056, 1058)} & \parallel & \underbrace{[1058, 1474)} \\ \text{256 challenge bits} & & \text{1024 signed low bytes} & & \text{두 size delta} & & \text{HBZ/h payload와 padding} \end{array}$$

이다. 모든 bit word가 0 또는 1이고 모든 low word가 signed-byte 범위에 있는 정준 입력(canonical input)의 집합(set)을 C_{prefix} 라 하자. 현재 실제 prefix packer와 unpacker에 대해

$$\text{Dec}_{\text{prefix}} \circ \text{Enc}_{\text{prefix}} = \text{id}_{C_{\text{prefix}}}$$

가 부분정확성(partial correctness)으로 증명되었다.

여기서 정의역 제한(domain restriction)은 장식이 아니다. noncanonical word에 대해서는 encode가 정보를 버리거나 decode가 다른 대표자(representative)를 돌려줄 수 있다. 대수학의 언어로는 C_{prefix} 가 선택된 대표자계(system of representatives)이고, round trip은 그 대표자계 위의 단면-수축 관계(section-retraction relation)다.

전체 서명 공간(signature space)에 대해서는 아직 이 등식(equality)이 없다. size metadata, HBZ와 h payload, canonical zero padding, 전체 parser의 성공과 유일성이 남아 있다.

2.5 어떤 대수적 사실이 어느 층에 있는가

대상	필요한 명제(proposition)	현재 상태
R_q -모듈(module) KeyGen	Jasmin 연산과 논문 KeyGen 분포(distribution)의 동일성	PARTIAL / 최상위 목표
packed key	$\pi_{992}(sk) = \pi_{992}(vk)$	PROVED (partial correctness)
raw μ transcript	Sign 64-byte 출력의 첫 32바이트와 Verify 32바이트 일치	PROVED (generated raw/internal)
challenge suffix	같은 position 64에서 μ_{32} 흡수 뒤 상태 일치	PROVED (국소 zero-loss)
signature prefix	canonical 입력에서 decode-after-encode가 항등 (identity)	PROVED (prefix 만)
HBZ/rANS suffix	실제 full encoder와 decoder의 역관계(inverse relation)	PARTIAL (actual core inverse는 PROVED , production full wrapper는 미완)
전체 서명 스킴	KeyGen/Sign/Verify refinement와 보안 합성	SPECIFIED / BLOCKED

다음 절은 이 표의 “부분정확성”, “국소”, “generated”, “실제”가 어떤 논리적 차이를 갖는지 설명한다.

3 표현 사상(representation map)과 프로그램 논리(program logic)의 번역 사전

3.1 배열과 메모리는 서로 다른 대상이다

EasyCrypt 추출물에는 고정 길이 배열과 전역 메모리가 함께 등장한다. 개념적으로

$$\text{Byte} = \{0, 1, \dots, 255\}, \quad \text{BArray}_N \simeq \text{Byte}^N$$

이고, 전역 메모리는 64비트 주소를 갖는 함수(function)처럼 읽을 수 있다.

$$\text{Mem} \simeq (\mathbb{Z}/2^{64}\mathbb{Z}) \rightarrow \text{Byte}.$$

실제 모델에는 load/store 연산과 word 표현이 있으므로 단순한 set-theoretic 함수보다 구조가 많지만, 구간 등식(interval equality)을 읽는 데에는 이 관점이 충분하다.

로컬 배열의 i 번째 원소와 메모리 base 주소 뒤 i 번째 바이트가 같다는 관계(relation)는

$$\forall, 0 \leq i < n, \quad a_i = \text{Mem}(\text{base} + i)$$

풀이다. Mode2KeyMemoryBridge.ec와 ApiKeyMemoryBridge.ec는 이 관계를 실제 importer/exporter 절차에 연결한다. 배열 등식만 증명해 두고 public pointer에 대한 결론을 말할 수 없는 이유가 여기에 있다.

정의 3.1 (구간 제한(interval restriction)과 region equality). 주소구간 $U = [b, b + n)$ 에 대해 $m \upharpoonright_U$ 를 그 구간의 바이트 함수라 하자. 두 메모리의 국소 관계는

$$\text{RegionEq}(m_1, m_2; b_1, b_2, n) \iff \forall, 0 \leq i < n, m_1(b_1 + i) = m_2(b_2 + i)$$

로 읽는다. base 주소가 같을 필요도, 구간 밖 메모리가 같을 필요도 없다.

Week 6의 region-local μ 정리(theorem)는 전체 메모리 등식 대신 key, pre, message 구간의 이 관계만 요구한다. 이것은 hash 절차가 실제로 읽는 부분에 맞춘 최소 관계다.

3.2 주소의 두 표현과 정준 범위(canonical range)

raw ABI는 정수 주소를 쓰는 부분과 W64 word 주소를 쓰는 부분을 모두 포함한다. 사상(map)

$$\mathbb{Z} \xrightarrow{\text{of_int}} \mathbb{Z}/2^{64}\mathbb{Z} \xrightarrow{\text{to_uint}} [0, 2^{64}) \cap \mathbb{Z}$$

은 모든 정수(integer)에서 항등(identity)이 아니다. wraparound를 제외한 정준 범위(canonical range)에서만 $\text{to_uint}(\text{of_int}(a)) = a$ 다.

따라서 **OBL-API-ADDRESS-BINDING**의 **PROVED**는 메모리 안전성 전체가 아니라 두 주소 표현이 같은 바이트를 가리킨다는 제한된 bridge다. valid region과 no-wrap 전제는 정리(theorem)의 정의역(domain)을 나타내며, 제거할 수 있는 구현 세부사항이 아니다.

3.3 절차는 전역 함수(total function)가 아니라 부분적 상태전이(partial state transition)다

결정론적 절차 f 를 상태공간(state space) X 에서 Y 로 가는 사상(map)으로 쓰고 싶지만, 재시도나 무한 루프 가능성이 있으면 자연스러운 대상은 부분함수(partial function)

$$f : X \rightharpoonup Y$$

또는 종료하는 실행만 포함하는 관계(relation)다. Hoare 명제(proposition)

$$\{P\}f\{Q\}$$

는 이 문서에서 다음처럼 읽는다.

$$\forall x \in X, \quad P(x) \wedge f(x) \downarrow \implies Q(x, f(x)).$$

여기서 $f(x) \downarrow$ 는 실행이 반환한다는 뜻이다. 이 명제(proposition)만으로 $f(x) \downarrow$ 를 증명하지 않는다.

번역 원리 3.2 (Hoare 정리(theorem)). $\text{hoare}[P:\text{pre} \implies \text{post}]$ 는 “pre가 정의하는 부분공간(subspace)에서 실제 절차가 종료할 때 그 그래프(graph)가 post 부분집합(subset)에 포함된다”로 읽는다. KeyGen 접두부 정리(theorem)와 codec round trip은 이 형태다.

두 프로그램 f_1, f_2 의 관계적 정리(relational theorem)는

$$\text{equiv}[f_1 \sim f_2 : P \implies Q]$$

형태로 나타난다. 두 초기 상태가 P 로 관련되면 두 종료 실행의 결과 상태가 Q 로 관련된다는 pRHL 명제(proposition)다. 좌우 상태의 변수에는 EasyCrypt 표기 $x\{1\}, x\{2\}$ 가 붙는다.

번역 원리 3.3 (관계적 정리(relational theorem)). pRHL 정리(theorem)는 보통 “두 상태기계의 시뮬레이션(simulation)” 또는 “관측량에 대한 쌍대시뮬레이션(bisimulation) 조각”으로 읽을 수 있다. 다만 현재 파일이 한 방향의 결과 관계만 증명한다면 완전한 bisimulation이라는 용어를 쓰지 않는다.

3.4 정제(refinement)는 가환사각형(commutative square)이다

추상 연산(abstract operation) F , 실제 절차 f , 입력·출력 표현사상(representation map) e_X, e_Y 가 있을 때 전형적인 목표는

$$\begin{array}{ccc} X & \xrightarrow{F} & Y \\ \downarrow e_X & & \downarrow e_Y \\ X_{\text{bytes}} & \xrightarrow{f} & Y_{\text{bytes}} \end{array} \quad e_Y \circ F = f \circ e_X$$

이다. 실제 저장소에서는 한 번에 이 전체 사각형을 닫기보다 다음처럼 잘게 분해한다.

1. 순수 배열 불변식(invariant) 또는 정수 항등식(integer identity);
2. 추출된 leaf 절차에 대한 Hoare 정리(theorem);
3. generated wrapper가 leaf와 같은 결과를 준다는 exact adapter;
4. raw API의 importer/exporter와 주소 bridge;
5. 실제 caller의 분기와 메모리 frame;
6. 최상위 기능·확률·보안 합성.

각 단계는 서로 다른 상태공간(state space)을 사용한다. 따라서 아래 단계의 정리(theorem)가 위 단계의 전제를 실제로 산출하는지 확인하지 않으면 합성할 수 없다.

3.5 프레임(frame)은 쓰기 지지집합(write support)에 대한 정리(theorem)다

절차가 메모리 구간 W 만 쓴다고 할 때 frame 성질은 보통

$$a \notin W \implies m_{\text{after}}(a) = m_{\text{before}}(a)$$

또는 관심 구간 U 가 W 와 서로소(disjoint)라는 전제 아래

$$m_{\text{after}} \upharpoonright U = m_{\text{before}} \upharpoonright U$$

로 표현된다. Week 6의 실제 Sign output frame은 1474바이트 signature 쓰기와 8바이트 signature-length 쓰기가 분리된 VK/SK/pre/message 구간을 보존함을 보인다. 이 성질이 없으면 Sign 뒤 Verify에서 같은 key/message를 읽는다고 합법적으로 말할 수 없다.

비유(형식화된 정리(theorem) 아님).

frame rule은 함수의 지지집합(support)이 W 에 들어간다는 명제(proposition)와 비슷하다. 관심 대상이 U 에 제한되어 있고 $U \cap W = \emptyset$ 이면 제한사상(restriction map) 아래 변화가 보이지 않는다. 실제 증명은 support나 sheaf 용어 대신 byte load/store의 점별 등식(pointwise equality)을 사용한다.

3.6 정준성(canonicity)은 대표자(representative) 선택이다

codec 정리(theorem)에는 다음 세 종류의 전제가 반복된다.

- challenge word가 정확히 0 또는 1이다;
- signed low coefficient가 지정된 byte 범위에 있다;
- HBZ coefficient가 $[-6, 7]$ 에 있다.

이들은 decode가 encode의 왼쪽 역(left inverse)이 되도록 하는 대표자 조건이다. 일반적으로 encode는 더 큰 word 공간에서 바이트 공간으로 가며 정보를 버릴 수 있으므로, 전 정의역에서 단사(injective)일 필요가 없다. 정준 부분집합(canonical subset) C 에서

$$\text{Dec} \circ \text{Enc} \upharpoonright_C = \text{id}_C$$

를 증명하는 것이 정확한 목표다. 여기서 오른쪽은 항등사상(identity map)이다. 또한 출력 배열의 사용하지 않는 tail이 보존되는지, padding이 유일한지, parser가 실패하지 않는지는 별도 frame과 canonical parsing 정리(theorem)다.

3.7 exact trace와 ghost observation

실제 raw caller는 hash 호출 전후에도 많은 계산을 수행한다. 분석을 위해 프로젝트는 실제 결과와 최종 메모리를 보존하면서 중간 값 *observed_mu*, 분기 도달 여부 *tail_reached*, 실제 hash descriptor 입력을 ghost field에 기록하는 trace 모듈을 둔다.

exact trace 정리(theorem)는 대략

$$\text{obs}(\text{actual execution}) = \text{obs}(\text{instrumented trace})$$

를 실제 반환값과 메모리까지 포함해 증명한다. 그러나 ghost field를 정의했다고 그 값의 수학적 관계가 자동으로 생기지는 않는다. 특히 서로 다른 두 완료된 trace의 *observed_mu* 등식은 반환값 수준의 pRHL 정리(theorem)를 trace 사후상태로 운반하는 별도 product/replay 논리를 요구한다.

3.8 “zero-loss”의 정확한 범위

결정론적인 두 generated 절차가 관련 입력에서 같은 관측값을 만든다는 pRHL 정리(theorem)가 있으면 그 국소 *seam*의 통계적 손실(statistical loss)을 0으로 둘 수 있다. 현재

$$\delta_{\mu, \text{raw_top}} = 0$$

라고 요약할 수 있는 이유가 이것이다. 그러나 다음 승격은 허용되지 않는다.

$$\delta_{\mu, \text{raw_top}} = 0 \not\Rightarrow \delta_{\text{Sign}} = \delta_{\text{Verify}} = \delta_{\text{Encoding}} = 0.$$

더 큰 δ 에는 sampler 분포(distribution), rejection/termination, 전체 codec, 실제 API composition, challenge의 나머지 입력 등이 모두 들어간다.

3.9 EasyCrypt 정리(theorem) 하나를 수학 문장으로 읽는 예

ExactMuTopControl.ec의 *sign_verify_generated_raw_mu_prefix*는 실제 generated 절차 *_sf_mu_rawpre*와 *__verify_hash_mu*를 직접 비교한다. 그 전제(premise)를 수학적으로 묶으면 다음과 같다.

1. 두 실행이 같은 global-memory 관계(relation)와 같은 pre/message descriptor를 갖는다.
2. Sign의 local secret-key 배열 첫 992바이트와 Verify의 memory key region이 같다.
3. 주소구간은 canonical하며 wraparound가 없다.
4. Verify의 prelength는 실제 raw branch를 선택한다.

결론은

$$\text{take}_{32}(\text{return}_{\text{Sign}}) = \text{return}_{\text{Verify}}$$

이다. 이 문장에서 “실제 generated”, “반환값”, “raw branch”, “32바이트 접두부” 중 하나라도 빼면 정리(theorem)를 강하게 잘못 읽게 된다.

4 현재 닫힌 증명 사슬

이 절의 “검증됨”은 2026년 8월 10일 스냅샷의 67개 authored target을 대상으로 한다. 개수는 프로젝트의 크기를 나타낼 뿐, 아래 부모 주장 모두가 완료되었다는 뜻은 아니다. 각 명제(proposition)의 정확한 전제와 EasyCrypt 이름은 [appendix A](#)에서 찾을 수 있다.

4.1 두 개의 주 진행선

현재 작업은 서로 연결되지만 상태가 다른 두 진행선으로 보는 것이 가장 쉽다.

1. **key/transcript 선**: packed key 접두부에서 raw μ 와 challenge suffix까지 내려가고, 실제 API memory/trace로 올리는 선;
2. **signature codec 선**: canonical signature prefix와 HBZ/rANS suffix의 역함수(inverse function)를 실제 pack/unpack 절차까지 달는 선.

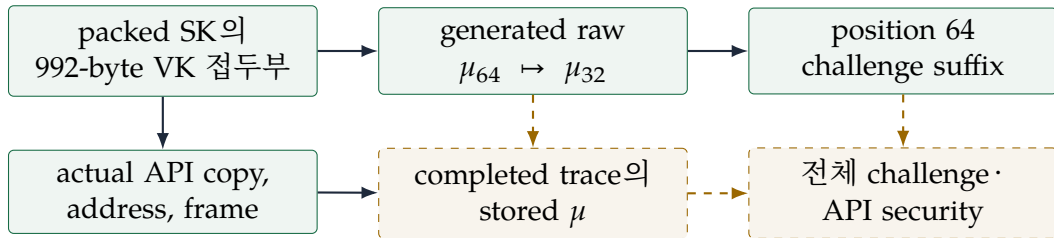


그림 2: key/transcript 진행선. 실선은 닫힌 정리(theorem), 점선은 열린 승격이다.

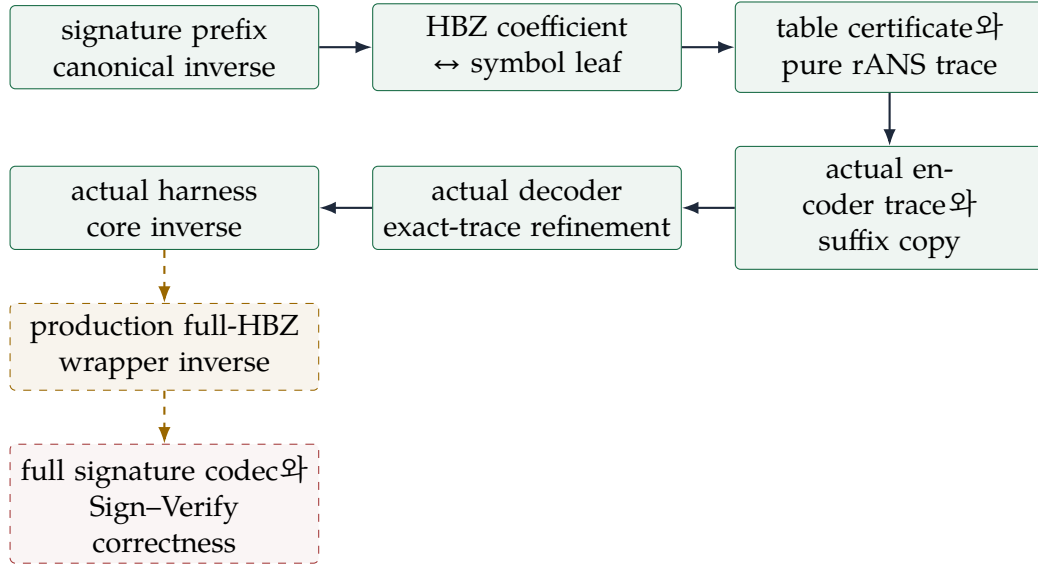


그림 3: signature codec 진행선. Week 11은 actual encoder의 성공 접미부(success suffix)를, Week 12는 exact trace를 소비하는 actual decoder를, Week 13은 actual copy를 사이에 둔 core 합성(composition)을 달았다.

4.2 1단계: packed secret key의 검증키 몫(quotient)

순수 배열 수준에서 술어

$$\text{VKPrefix}_{992}(sk, vk) \iff \forall, 0 \leq i < 992, sk_i = vk_i$$

를 둔다. PackedKeyPrefix.ec는 다음 기본 사실을 증명한다.

- 한 바이트씩 복사하는 loop invariant가 다음 index로 보존된다;
- 992바이트를 모두 복사하면 prefix relation이 성립한다;
- index 992 이후의 쓰기는 이미 완성된 접두부를 바꾸지 않는다.

이 배열 보조정리(lemma)는 실제 추출 절차로 올라간다.

검증된 명제(verified proposition) 4.1 (실제 packer의 접두부). pack_sk_m23_mode2_vk_prefix는 mode 2 상수 아래 실제 generated _pack_sk_m23가 반환할 때 결과 secret-key 배열의 첫 992바이트가 입력 verification-key 배열과 같음을 보인다. 뒤이어 eta-vector와 key를 쓰는 루프가 이 접두부를 보존한다.

검증된 명제(verified proposition) 4.2 (internal KeyGen 반환). keypair_full_m23_mode2_return_prefix와 keypair_internal_mode2_return_prefix는 위 관계를 실제 generated KeyGen 반환값까지 올린다.

$$\text{KeyGen returns } (vk, sk) \implies \text{VKPrefix}_{992}(sk, vk).$$

두 정리(theorem)는 **PROVED** partial correctness다. KeyGen rejection loop의 종료, 전체 출력 분포, 논문 KeyGen과의 동일성을 증명하지 않는다. 따라서 **FC-KG-TOP**은 여전히 **PARTIAL**이다.

4.3 2단계: 로컬 배열에서 public memory까지

KeyGen이 반환한 배열 관계를 Sign/Verify의 실제 API 입력에 사용하려면 네 종류의 화살표가 필요하다.

1. KeyGen local VK/SK 배열을 외부 주소로 복사하는 exporter;
2. Sign이 외부 SK를 local 배열로 가져오는 importer;
3. Verify가 외부 VK를 local 배열로 가져오는 importer;
4. 정수 주소와 W64 주소가 같은 구간을 나타낸다는 canonical bridge.

ApiKeyMemoryBridge.ec는 실제 copy helper에 대해 mode 2의 992/1408바이트 prefix와 disjoint frame을 증명한다.

검증된 명제(verified proposition) 4.3 (실제 raw KeyGen의 순차 export). keypair_raw_api_exports_matching_prefixes는 실제 cryptolab_haetae_mode2_keypair_internal이 반환할 때, 명시적인 valid/canonical/disjoint region 계약 아래 외부 SK와 VK 구간의 첫 992바이트가 같음을 보인다.

이 단계는 단순히 “copy 함수가 복사한다”보다 강하지만 메모리 안전성 전체보다 약하다. pointer validity와 separation은 전제로 드러나며, KeyGen 종료성은 여전히 결론이 아니다.

4.4 3단계: Sign과 Verify의 generated raw μ

Sign은 local packed SK의 접두부를 흡수하고 64바이트 μ 를 squeeze한다. Verify는 global memory의 VK 구간을 흡수하고 32바이트를 squeeze한다. 실제 generated helper chain은 초기화, key/pre/message absorb, Keccak permutation, finalize, squeeze를 서로 다른 절차 표면으로 실행한다.

ExtractedMuHashPrefix.ec가 leaf 관계들을 증명하고, ExactMuTopControl.ec가 실제 top 절차로 올린다.

검증된 명제(verified proposition) 4.4 (generated raw μ 접두부). 명시적인 mode 2 key-prefix, 같은 pre/message 구간, raw-branch, valid/no-wrap 전제 아래

$$\text{SignRawMu} \sim \text{VerifyHashMu}$$

가 성립하며 결론은

$$\text{take}_{32}(\mu_{\text{Sign}}^{64}) = \mu_{\text{Verify}}^{32}$$

이다. 명명된 정리(theorem)는 sign_verify_generated_raw_mu_prefix다.

이 정리(theorem)는 두 실제 generated 절차를 theorem surface에 직접 둔다. wrapper에 대한 추상 명제(abstract proposition)를 실제 절차와 동일시해 버린 것이 아니다. 또한 raw_prelen_shr63_zero와 raw_prelen_branch_guard가 실제 bit-level 분기로 raw 경로를 선택함을 보인다.

4.5 4단계: 전체 메모리 등식(whole-memory equality)에서 read-region 등식(read-region equality)으로

실제 Sign이 signature를 출력한 뒤에는 두 실행의 global memory 전체가 같을 이유가 없다. 필요한 것은 hash가 읽는 key, pre, message 구간뿐이다.

검증된 명제(verified proposition) 4.5 (region-local generated hash). sign_verify_generated_raw_mu_prefix_regionwise는 서로 다른 global memory를 허용한다. key/pre/message read region이 점별로 관련되면 실제 generated Sign/Verify raw hash 반환값이 같은 32바이트 접두부를 갖는다.

검증된 명제(verified proposition) 4.6 (Sign 출력 frame). sign_raw_api_frames_reused_regions는 실제 raw Sign이 signature 1474바이트와 siglen 8바이트를 쓰더라도, 출력 구간과 분리된 VK/SK/pre/message 구간을 보존함을 보인다.

두 정리(theorem)를 합치면 “Sign 뒤에도 Verify가 읽을 입력 바이트가 남아 있다”는 memory-level 기반이 생긴다. 하지만 완료된 두 trace가 저장한 μ 를 직접 비교하는 마지막 운송은 아직 열려 있다. 이는 [section 6](#)에서 분리한다.

4.6 5단계: position 64 challenge suffix

Sign과 Verify가 같은 challenge state와 position을 갖고 position이 64이며, 앞 단계에서 얻은 μ_{32} 가 같다고 하자. 실제 generated absorb helper를 한 번씩 실행한 뒤 상태와 position이 같다는 정리(theorem)가 있다.

검증된 명제(verified proposition) 4.7 (국소 challenge-suffix zero-loss). generated_raw_mu_to_challenge_suffix_zero_loss는 실제 generated raw μ 절차와 실제 generated 32-byte challenge absorb helper를 각 변에 연속 호출한다. 명시된 전제 아래 최종 $(state, position)$ 이 같다.

따라서 raw/internal generated seam에서는 deterministic loss가 0이다. 하지만 이 정리(theorem)는 highbits/LSB prefix, 전체 challenge, 실제 accepted API trace의 stored μ , context branch를 포함하지 않는다.

4.7 6단계: 실제 caller의 control과 accepted Verify

Week 4-6은 위 leaf 결과를 실제 public/raw caller 문맥에 배치하는 작업을 했다. 현재 다음이 각각 컴파일된다.

- 실제 raw Sign의 exact μ trace와 hash input binding;

- 실제 raw/cryptolab Verify의 모든 early-reject/tail branch를 보존하는 exact trace;
- Verify가 accept하면 실제 tail과 hash call에 도달한다는 방향;
- accepted Verify가 실제 VK/pre/message descriptor를 사용했다는 binding;
- 실제 Sign 다음 실제 Verify를 순차 실행하는 exact trace;
- 그 순차 실행 동안 key/pre/message 구간을 보존하는 frame.

검증된 명제(verified proposition) 4.8 (accept에서 tail로).

$$VerifyResult = 0 \implies tail_reached$$

가 실제 raw/cryptolab Verify에 대해 증명되었다. 역방향도 아니고, “임의의 1474바이트 signature가 tail에 도달한다”는 명제(proposition)도 아니다.

검증된 명제(verified proposition) 4.9 (실제 sequential trace). raw_sign_then_verify_actual_exact_trace는 실제 raw Sign 후 실제 raw Verify의 반환값과 최종 메모리를 두 instrumented trace의 순차 실행과 맞춘다.

그럼에도 **OBL-DIRECT-OBSERVED-MU**는 **PARTIAL**이다. exact trace는 관측 위치와 값을 노출하지만, 별도의 pRHL 반환값 정리(theorem)를 이미 종료한 두 trace의 ghost field 등식(equality)으로 자동 윤반하지 않는다.

4.8 7단계: 실제 signature prefix의 section-retraction

Mode2SignaturePrefixPack.ec와 Mode2SignaturePrefixUnpack.ec는 실제 generated prefix loop의 정확한 layout을 연다. 두 절차를 실제 순서로 호출하는 harness가 Mode2SignaturePrefixRoundTrip.ec에 있다.

검증된 명제(verified proposition) 4.10 (mode 2 prefix round trip). canonical challenge와 canonical signed-low 입력에 대해 pack_unpack_sig_prefix_mode2_roundtrip은

$$\begin{aligned} decodedChallenge[0..256) &= challenge[0..256), \\ decodedLow[0..1024) &= low[0..1024) \end{aligned}$$

를 보이고, decoded-low의 사용하지 않는 tail도 frame한다.

정준 전제(canonical premise)의 all-zero witness도 컴파일되어 명제(proposition)가 공허하지 않다. 그러나 결과는 서명 앞 1056바이트뿐이다. metadata 두 바이트와 416바이트 suffix, padding, 전체 unpack 성공은 포함되지 않는다.

4.9 8단계: HBZ coefficient-symbol leaf

HBZ 입력은 1024개 coefficient $h_i \in [-6, 7)$ 다. prepare는 $s_i = h_i + 6 \in \{0, \dots, 12\}$ 을 계산하고, apply는 $s_i - 6$ 을 32비트 word 표현으로 복원한다.

검증된 명제(verified proposition) 4.11 (HBZ leaf inverse). encode_prepare_decode_apply_mode2_inverse는 canonical HBZ 배열에 대해 실제 prepare와 실제 apply 절차를 순차 실행하면 앞 1024개 coefficient가 복원되고, decoder coefficient tail이 보존되며, prepare의 bad 출력이 0임을 보인다.

이 정리(theorem)는 rANS core를 건너뛴 leaf-level inverse다. 따라서 이것만으로 실제 compressed suffix가 round trip한다고 말할 수 없다.

4.10 9단계: 실제 표와 순수 rANS 수학

mode 2 HBZ의 13개 frequency는 합이 1024이고, 누적구간은 $[0, 1024)$ 를 분할(partition)한다. Mode2HbzTableCertificate.ec는 이 산술뿐 아니라 reciprocal/bias와 packed decoder word의 의미를 증명한다. 생성된 실제 literal array가 그 certificate를 만족한다는 정리(theorem)도 별도 파일에서 컴파일된다.

검증된 명제(verified proposition) 4.12 (순수 단일 단계 역함수(inverse function)). 각 symbol s 와 허용된 state x 에 대해 실제 table에서 얻은 c_s, f_s 를 사용하면

$$D_s(E_s(x)) = x$$

이고, reciprocal을 쓰는 fast encoder step도 명시된 normalized range에서 수학적 E_s 와 같다.

Week 9은 단일 단계 사이의 normalization byte를 포함하는 순수 $xs/cuts/bytes$ trace를 정의했다.

검증된 명제(verified proposition) 4.13 (순수 byte-stack). canonical symbol list에 대해 다음이 컴파일된다.

- normalization은 매 단계 0, 1, 2바이트만 방출한다;
- 방출된 segment를 decoder 순서로 append하면 원래 state를 복원한다;
- 4-byte little-endian 초기 state는 parse 뒤 복원된다;
- 모든 trace state는 $[2^{23}, 2^{31})$ 에 머문다;
- trace의 head symbol, reduced state, normalization segment가 한 단계씩 decode된다.

검증된 명제(verified proposition) 4.14 (실제 copy와 control harness). copy_encoded_suffix_correct는 actual __copy_encoded_suffix의 점별 slice copy와 tail frame을 보인다. actual_rans_harness_branches_on_encoder_result는 실제 encoder, 실제 copy, 실제 decoder를 이 순서로 호출하고 encoder의 실제 bad 결과가 0일 때 정확히 decoder를 실행하며 $(count, size, m) = (1024, size, 13)$ 을 전달함을 보인다.

이 control 정리(theorem) 자체는 decoder success, symbol equality, 최종 state, 소비 byte 수를 결론내리지 않는다. Week 12의 별도 decoder 정리(theorem)가 exact-trace 입력 관계(input relation) 아래에서 그 의미론(semantics)을 증명하지만, 이 시점에는 아직 control harness 안에 합성되지 않았다. Week 13의 actual_rans_encode_copy_decode_inverse가 같은 harness에 그 의미론적 결론(semantic conclusion)을 추가한다.

4.11 10단계: actual encoder의 국소 의미론(local semantics)

Week 10에는 rANS encoder를 위한 6개 타깃이 authored target manifest에 편입되어 fresh compile되었다. 이들은 순수 trace와 실제 생성 절차 사이의 한 거대한 정리(theorem)를 바로 선언하지 않고, 배열(array)-리스트(list), word 산술(word arithmetic), 내부 정규화(inner normalization), 최종 직렬화(final serialization)를 분리한다 [9, 6].

검증된 명제(verified proposition) 4.15 (배열(array), word, 직렬화(serialization) 보조정리(lemmas)). encode_trace_suffix_extension은 실제 1024-byte 기호 배열(symbol array)의 접미부(suffix)를 한 기호씩 확장하는 순수 trace 점화식(recurrence)으로 옮긴다. actual_mode2_encoder_word_step_correct는 literal table의 reciprocal, shift, bias, complement를 쓰는 실제 word 식이 순수 fast step과 같음을 보인다. actual_encoder_final_state_serialization은 네 번의 set8이 32-bit state의 little-endian 직렬화(serialization)와 같음을 보인다.

검증된 명제(verified proposition) 4.16 (실제 내부 루프(inner loop)와 성공 크기(success size)). `actual_rans_encode_inner_no_underflow`는 실제 `generated_rans_encode`의 중첩 정규화 루프(nested normalization loop)에 바이트 구간(byte segment)과 접두부 프레임(prefix frame) 불변식(invariant)을 설치하여 `W64 cursor 비언더플로(no-underflow)`를 보인다. `actual_rans_encode_success_size_bound`는 같은 절차가 반환한다면

$$bad \neq 0 \quad \vee \quad (bad = 0 \wedge 0 \leq off \leq 1020 \wedge 4 \leq 1024 - off \leq 1024)$$

임을 보이는 실제 부분정확성(actual partial correctness) 정리(theorem)다.

Week 10 단계의 끝에서 이 결과는 이전의 $bad \in \{0, 1\}$ 제어 정리(control theorem)보다 강했지만, `actual` 반환 접미부(returned suffix) 전체가 `TraceBytes(SymbolList(symbols0))`와 같다는 명제(proposition)는 아니었다. 당시 내부 phase는 보수적인 0.4 범위(range)를 사용했고, 실제 바깥 반복(outer iteration)의 한 기호 상태전이(state transition)와 마지막 네 store의 절차 수준 합성이 열려 있었다.

4.12 11단계: actual encoder의 전역 trace closure

Week 11은 7개 encoder 타깃을 추가하여 Week 10의 국소 전이(local transition)를 전체 성공 출력(global success output)으로 합성했다 [10, 6]. 핵심 불변식(invariant)은 임의의 inner-loop 시작 배열이 아니라 최초 배열 `enc0`와 이미 처리한 순수 접미부(pure suffix)를 함께 기억한다.

검증된 명제(verified proposition) 4.17 (tail-aware 불변식(invariant)과 생성 산술(generated arithmetic)). `encoder_inner_tail_inv` 명제(proposition)는 actual inner loop가

$$\text{InnerWrittenSuffix}(x_0, s, k) \text{ ++ EncodeTrace}(\text{tail}).\text{bytes}$$

를 현재 cursor 뒤에 보존한다고 표현한다. `encoder_inner_tail_exit_exact` 보조정리(lemma)는 실제 guard가 끝나면 $k = \text{NormalizationLen}(x_0, s) \leq 2$ 임을 회복한다. `generated_loaded_nested_word_update` 보조정리(lemma)는 실제 table load, reciprocal high product, shift, complement와 bias를 순수 fast step으로 운송한다.

검증된 명제(verified proposition) 4.18 (encoder trace closure 정리(theorem)). `actual_rans_encode_trace_closure`는 실제 `generated RansEncodeTarget.M._rans_encode`를 직접 열어, 반환한다면 실패하거나 성공 시

$$\begin{aligned} bad' = 0, \quad 0 \leq off' \leq 1020, \quad 4 \leq 1024 - off' \leq 1024, \\ \text{SegmentMatches}(enc', off', \text{TraceBytes}(\text{SymbolList}(\text{symbols}_0))), \\ off' + |\text{TraceBytes}(\text{SymbolList}(\text{symbols}_0))| = 1024, \quad \text{PrefixFrame}(enc_0, enc', off') \end{aligned}$$

가 모두 성립함을 보인다. `actual_rans_encode_trace_refinement`는 같은 결론을 부모 refinement 표면에 전개한 따름정리(corollary)다.

마지막 네 generated store가 직렬화된 상태(serialized state)를 누적 normalization suffix 앞에 붙인다는 합성도 이 정리(theorem) 안에 들어간다. 따라서 **OBL-RANS-ACTUAL-INNER-NORMALIZE**, **OBL-RANS-FINAL-SERIALIZATION**, **OBL-RANS-ENCODE-REFINEMENT**는 각각 명시된 범위에서 **PROVED**다. 그러나 이는 Hoare 부분정확성(Hoare partial correctness)이다. 실제 호출의 종료성(termination)이나 $bad' = 0$ 인 실행의 존재를 주지 않으므로 **OBL-RANS-ACTUAL-SUCCESS-WITNESS**는 **PARTIAL**이다.

4.13 12단계: actual decoder의 exact-trace 의미론(semantics)

Week 12는 8개 decoder 타깃을 추가하여 순수 trace의 좌표(coordinates)를 actual generated decoder의 두 중첩 반복(nested loops)에 설치했다 [11, 5]. 핵심 입력 술어(input predicate) `actual_mode2_decoder_trace_input`은 임의의 buffer가 아니라 다음을 동시에 요구한다.

- expected symbol array가 mode 2 정준 기호열(canonical symbol stream)임;
- $encoded_size = |\text{TraceBytes}(\text{SymbolList}(\text{expected}))|$ 이고 $4 \leq encoded_size \leq 1024$;
- $buffer_0[0..encoded_size)$ 가 그 exact trace와 점별로 일치함;
- generated normalization read마다 pure trace segment의 해당 byte를 읽음;
- actual state fields가 $(count, size, m) = (1024, encoded_size, 13)$ 임;
- 실제 mode 2 symbol_words와 dsyms_words를 사용함.

검증된 명제(verified proposition) 4.19 (decoder trace 좌표(coordinates)와 불변식(invariant)). $decoder_state_at$, $decoder_cursor$, $decoder_segment_at$ 은 순수 $xs/cuts/bytes$ trace의 상태(state), cursor, normalization segment를 iteration별 좌표로 만든다. $decoder_outer_trace_live$ 명제(proposition)는

$$x = \text{DecoderStateAt}(i), \quad off = \text{DecoderCursor}(i)$$

와 decoded prefix의 등식(equality), untouched tail frame을 함께 보존한다. 안쪽 불변식(inner invariant)은 해당 segment를 정확히 0/1/2-byte replay하고 다음 순수 상태(pure state)를 복원한다.

검증된 명제(verified proposition) 4.20 (actual decoder exact-trace 정리(theorem)). $actual_rans_decode_trace_refinement$ 는 실제 generated $\text{RansDecodeTarget.M.}_rans_decode$ 를 직접 열어, 반환한다면

$$\begin{aligned} bad' &= 0, & off' &= encoded_size, \\ \forall i, 0 \leq i < 1024 &\implies decoded'_i = expected_i, \\ \forall i, 1024 \leq i < 2048 &\implies decoded'_i = decoded_{0,i} \end{aligned}$$

를 보인다. 내부 종료 상태(internal exit state) $x = 2^{23}$ 도 유도하여 실제 final reject를 통과시키지만, generated ABI는 이 x 를 반환하지 않으므로 반환 배열(returned array)과 state fields만 사후조건(postcondition)에 공개된다.

따라서 **OBL-RANS-DEC-NORMALIZE**와 **OBL-RANS-DECODE-REFINEMENT**는 exact-trace Hoare 부분정확성(Hoare partial correctness)으로 **PROVED**다. Week 12 종료 시점에는 encoder의 $segment_matches$ 와 actual copy의 $slice_eq$ 를 위 입력 술어(input predicate)로 운송하는 sequential harness 정리(theorem)가 없었다. Week 13의 다음 단계가 정확히 이 간선(edge)을 닫는다.

4.14 13단계: actual encoder-copy-decoder core 역함수(inverse function)

Week 13은 2개 타깃을 추가하여 실제 $\text{Mode2RansActualHarness.run}$ 의 세 호출을 순서대로 합성했다 [12, 4]. 이 harness와 control 정리(theorem)는 $\text{Mode2RansActualInverse.ec}$ 에 있고, 의미론적 closure는 $\text{Mode2RansCoreCompositionBridge.ec}$ 와 $\text{Mode2RansCoreActualInverse.ec}$ 에 분리되어 있다. 새 bridge 파일은 다음 운송 보조정리(transport lemmas)를 제공한다.

- $copied_suffix_is_exact_trace$: encoder의 $segment_matches$ 와 actual copy의 $slice_eq$ 에서 copied buffer의 offset 0 exact trace를 얻음;
- $trace_bytes_trace_segment_nth$: 전역 $trace_bytes$ 좌표를 symbol별 local segment와 decoder cursor 좌표로 분해함;
- $segment_matches_implies_exact_decoder_segment_input$: global trace 구간(segment)에서 decoder의 모든 pointwise byte-read 전제를 얻고, $segment_matches_implies_decoder_word_reads$ 가 이를 actual W64 read 형태로 옮김;

- `actual_decoder_input_from_copied_trace` 와 `configured_decoder_state_fields`: copied trace와 실제 세 state store를 각각 decoder 입력 관계(input relation)의 구성요소로 만들;
- `actual_decoder_input_from_configured_trace`: copied trace, 정준 기호열(canonical symbol stream), size bound, 실제 세 state store에서 Week 12 decoder 입력 술어(input predicate) 전체를 구성함;
- `encoder_success_size_word_bridge`: actual (W64) 뺄셈과 정수 trace 크기(integer trace size)를 no-wrap 범위에서 동일시함.

검증된 명제(verified proposition) 4.21 (OBL-RANS-CORE-INVERSE). `actual_rans_encode_copy_decode_inverse`의 의미론적 전제(semantic precondition)는 harness argument binding과 `mode2_hbz_symbol_stream`뿐이다. encoder 성공, copied-trace 등식, decoder 실행 또는 성공을 전제로 요구하지 않는다. 반환한다면 다음 두 가지(branch) 중 하나다.

failure : $encoderBad \neq 0, \neg decoderRan, decoderOff = 0, decoderBad = 1,$
 $decoded' = decoded_0, decoderState' = decoderState_0;$

success : $encoderBad = 0, decoderRan, (count, size, m) = (1024, encodedSize, 13),$
 $decoderBad = 0, decoderOff = encodedSize,$
 $decoded'[0..1024) = expected[0..1024),$
 $decoded'[1024..2048) = decoded_0[1024..2048).$

성공 가지(success branch)는 actual encoder suffix relation과 초기 prefix frame도 보존한다. 따라서 세 actual generated 절차의 core 역함수(inverse function)가 성공 조건부 부분정확성(success-conditioned partial correctness)으로 컴파일된다.

`core_composition_preconditions_satisfiable`는 all-zero 정준 기호 배열(canonical symbol array)로 전제의 비공허성(non-vacuity)만 보인다. 결론이 실패/성공의 합(disjunction)이므로 실제 성공 가지의 도달가능성(reachability)이나 encoder/decoder 종료성(termination)은 따라오지 않는다. 따라서 **OBL-RANS-ACTUAL-SUCCESS-WITNESS**와 production full-HBZ wrapper는 **PARTIAL**로 남는다.

4.15 닫힌 사슬의 정확한 끝점

사슬	끝점
packed key $\rightarrow$ generated raw $\mu \rightarrow$ position-64 absorb	PROVED (raw/internal, 국소)
actual KeyGen export, Sign/Verify import, address/frame/control	PROVED (각 명명된 계약)
completed actual traces의 stored μ 직접 등식(direct equality)	PARTIAL
canonical signature prefix round trip	PROVED (1056-byte prefix)
HBZ leaf, table, pure rANS step와 byte trace	PROVED (순수/leaf)
actual encoder array/list $\cdot$ word $\cdot$ serialization leaf	PROVED (Week 10 국소 명제 (local propositions))
actual encoder 내부 byte/frame와 성공 크기	PROVED (exact 0..2, actual partial correctness)

사슬	끝점
actual encoder 전체 접미부(trace suffix) refinement	PROVED (성공 조건부 부분정확성(success-conditioned partial correctness))
actual decoder exact-trace refinement	PROVED (1024-symbol 복원, exact consumption, tail frame)
actual encoder-copy-decoder harness control	PROVED (호출·분기만)
actual encoder-copy-decoder의 inverse	PROVED (성공 조건부 부분정확성(success-conditioned partial correctness))
production full-HBZ wrapper inverse	PARTIAL (success-conditioned wrapper frame/size transport 미완)
actual encoder success witness	PARTIAL (concrete all-6 입력의 성공 도달성과 종료성 부재)
full signature codec와 Sign-Verify correctness	BLOCKED
구현 수준 기능정확성과 보안	SPECIFIED / PARTIAL

5 현재 전선: HBZ/rANS를 유한 동역학계(finite dynamical system)로 읽기

5.1 왜 production full-HBZ wrapper가 지금의 단일 gate인가

signature prefix의 역함수(inverse function), HBZ coefficient-symbol leaf, 그리고 actual encoder가 정준 기호열(canonical symbol stream)을 순수 byte trace로 보내는 간선(edge), exact trace를 actual decoder가 정준 기호열로 되돌리는 간선(edge), actual copy를 사이에 둔 core 합성(composition)은 각각 닫혔다. GO-CORE는 세 actual generated 절차의 성공 조건부 역함수(inverse function)가 컴파일되었다는 판정이다. full signature codec으로 가는 현재 단일 우선순위는 actual `_encode_hb_z1_full`과 `_decode_hb_z1_full`을 prepare/apply 역함수(inverse function) 및 증명된 rANS core와 합성하는 성공 조건부 full-HBZ wrapper 정리(theorem)이다 [12, 4].

rANS 자체의 정수 산술(integer arithmetic)은 복잡하지 않다. 어려움은 encoder가 symbol을 뒤에서부터 읽고 byte buffer cursor를 감소시키는 반면, decoder는 복사된 buffer를 앞에서부터 읽는다는 구현 방향성에 있다. 순수 역함수 정리(pure inverse-function theorem)를 실제 두 중첩 loop의 state와 결합하는 불변식(invariant)이 핵심이다. Week 11은 reverse encoder 쪽 결합을, Week 12는 forward decoder 쪽 결합을, Week 13은 두 정리(theorem) 사이의 actual copy와 전제 운송(premise transport)을 완료했다. 이제 남은 간극(gap)은 이 core 정리(theorem)를 `production _encode_hb_z1_full/_decode_hb_z1_full`의 prepare/apply·frame·size 의미론(semantics)과 합성하는 wrapper 경계다.

5.2 13-symbol 분할(partition)

HBZ 정준 계수(canonical coefficient) $h_i \in [-6, 7]$ 를 $s_i = h_i + 6$ 으로 옮기면 alphabet은

$$\Sigma = \{0, 1, \dots, 12\}$$

이다. rANS scale을 $M = 1024$ 라 하고 각 symbol의 누적 시작점과 빈도(frequency)를

$$\begin{aligned} (c_s)_{s=0}^{12} &= (0, 1, 2, 3, 8, 66, 312, 710, 957, 1016, 1021, 1022, 1023), \\ (f_s)_{s=0}^{12} &= (1, 1, 1, 5, 58, 246, 398, 247, 59, 5, 1, 1, 1) \end{aligned}$$

로 둔다. certificate는

$$f_s > 0, \quad \sum_{s=0}^{12} f_s = M, \quad [c_s, c_s + f_s) \text{가 } [0, M) \text{를 분할함}$$

을 보인다.

이 분할은 residue slot $y \bmod M$ 에서 symbol을 유일하게 복원하는 유한 분할(finite partition)이다. 실제 encoder/decoder의 packed tables가 정확히 이 c_s, f_s 와 symbol lookup을 나타낸다는 사실도 generated literal arrays에 대해 입증된다.

5.3 순수 단계는 유클리드 나눗셈(Euclidean division)의 역함수(inverse function)다

고정 기호(fixed symbol) s 에 대해

$$E_s(x) = \left\lfloor \frac{x}{f_s} \right\rfloor M + c_s + (x \bmod f_s),$$

$$D_s(y) = \left\lfloor \frac{y}{M} \right\rfloor f_s + (y \bmod M) - c_s$$

로 둔다. 유클리드 나눗셈(Euclidean division) $x = f_s \lfloor x/f_s \rfloor + (x \bmod f_s)$ 와 $0 \leq x \bmod f_s < f_s$ 에서 즉시

$$D_s(E_s(x)) = x$$

를 기대할 수 있다. EasyCrypt의 pure_rans_step_quotient, pure_rans_step_slot, pure_rans_step_inverse가 이 계산을 분리해 증명한다.

실제 encoder는 나눗셈 대신 table의 reciprocal, shift, bias를 사용하는 fast formula를 쓴다. 별도 certificate는

$$1 \leq x < x_{\max}(s), \quad x_{\max}(s) = 2^{21} f_s$$

에서 fast quotient가 유클리드 몫(Euclidean quotient)과 같고, 따라서 fast step도 D_s 에 의해 역산됨을 보인다. 이것은 단순히 “표 값이 맞아 보인다”는 계산 검사가 아니라 실제 word field의 정수 의미를 연결한 정리(theorem)다.

검증된 명제(verified proposition) 5.1 (순수 및 fast step). 정준 기호(canonical symbol) $0 \leq s < 13$ 과 명시된 상태 범위(state range)에서

$$D_s(E_s(x)) = x, \quad D_s(E_s^{\text{fast}}(x)) = x$$

가 컴파일된다. 전제 $s = 0, x = 1$ 의 concrete witness도 있어 정리(theorem)는 공허하지 않다.

5.4 정규화 바이트(normalization byte)는 잃어버린 몫(quotient)의 좌표(coordinate)다

state가 $x_{\max}(s)$ 보다 크면 fast step을 적용하기 전에 byte radix $B = 256$ 로 0, 1, 2회 나누어 state를 줄인다. 순수 정의는

$$\ell_s(x) \in \{0, 1, 2\}, \quad r_s(x) = \left\lfloor \frac{x}{B^{\ell_s(x)}} \right\rfloor$$

이고, 나눌 때 버린 낮은 byte들을 decoder가 읽을 순서의 list $\beta_s(x)$ 로 보존한다.

핵심 readback 명제(proposition)는

$$\text{ReadBytes}(r_s(x), \beta_s(x)) = x.$$

한 byte일 때는 $x = B \lfloor x/B \rfloor + (x \bmod B)$ 이고, 두 byte일 때는 이 항등식(identity)을 두 번 적용한다. 실제 encoder는 낮은 byte를 먼저 쓰면서 cursor를 감소시키므로 최종 forward segment에서 두 byte의 순서는 [high, low]다. 이 방향 전환을 놓치면 순수 정리(theorem)는 맞아도 actual decoder와 맞지 않는다.

Mode2RansNormalization.ec는 실제 W32 shift/truncate/or와 W64 cursor decrement가 이 정수 연산과 같은 의미를 갖는다는 leaf 정리(theorem)를 제공한다.

5.5 전체 symbol list의 순수 trace

symbol list를 $s_0, \dots, s_{n-1}$, $n = 1024$ 라 하고 마지막 state를 $x_n = L = 2^{23}$ 로 둔다. encoder가 뒤에서 앞으로 진행하도록

$$\begin{aligned}\ell_j &= \ell_{s_j}(x_{j+1}), \\ r_j &= \lfloor x_{j+1} / 256^{\ell_j} \rfloor, \\ x_j &= E_{s_j}^{\text{fast}}(r_j), \\ \beta_j &= \beta_{s_j}(x_{j+1})\end{aligned}$$

로 재귀적으로(recursively) 정의한다. decoder-facing byte string은

$$\mathcal{B} = \text{LE32}(x_0) \parallel \beta_0 \parallel \dots \parallel \beta_{n-1}$$

이다. cursor cut은

$$c_0 = 4, \quad c_{j+1} = c_j + |\beta_j|$$

로 둔다.

프로젝트의 이름 *xs/cuts/bytes*는 각각 $(x_0, \dots, x_n)$, $(c_0, \dots, c_n)$, $\mathcal{B}$ 를 가리킨다. 이 세 대상은 `Mode2RansByteStack.ec`에 정의되어 있다. 그 파일의 `valid_rans_trace`는 임의의 witness를 전제로 받지 않고 canonical symbol list로부터 이들을 직접 구성한다.

검증된 명제(verified proposition) 5.2 (트레이스(trace)의 귀납적 역산(inductive inversion)). canonical list에 대해

$$2^{23} \leq x_j < 2^{31}$$

가 모든 j 에서 성립한다. 또한 x_j 의 residue slot은 s_j 를 선택하고, $D_{s_j}(x_j) = r_j$ 이며, β_j 를 읽으면 x_{j+1} 가 복원된다.

따라서 순수 decoder의 리스트 수준 귀납증명(list-level inductive proof)에 필요한 수학적 내용은 준비되어 있다. actual encoder state가 이 trace 좌표를 나타낸다는 refinement는 Week 11에, actual decoder state가 같은 좌표를 순방향(forward)으로 소비한다는 refinement는 Week 12에 달했다.

5.6 Week 10에서 감사된 여섯 encoder 타깃

Week 10은 필요한 바깥 루프 불변식(outer-loop invariant)을 서로 다른 운송 문제로 분해한 다음 6개 파일을 authored target manifest에 편입했다. 여섯 파일은 모두 `-no-eco`로 fresh compile 되지만, 각 국소 명제(local proposition)의 완료와 부모 refinement의 완료는 구분해야 한다.

- `Mode2RansArrayListBridge.ec`: actual byte array의 symbol stream을 순수 list와 suffix로 옮기고, segment 일치와 prefix frame을 표현한다. 배열(array)-리스트(list) 다리와 한 기호 trace 점화식(recurrence)은 **PROVED**다.
- `Mode2RansEncoderWordStep.ec`: packed reciprocal, shift, complement를 쓰는 실제 word 식을 순수 fast step의 정수 의미와 연결한다. literal mode 2 table에 대한 word-level 등식(equality)은 **PROVED**다.
- `Mode2RansEncoderInnerProgress.ec`: normalization 내부 loop의 순수한 0/1/2회 진행을 state, cursor, 이미 쓴 byte segment로 좌표화한다. 이 순수 보조정리(pure lemma)는 **PROVED**다.
- `Mode2RansEncoderSerialization.ec`: 마지막 W32 state의 little-endian 네 바이트, 바깥 구간 frame, 성공 시 size 산술을 분리한다. 이 leaf 명제(leaf proposition)는 **PROVED**다.

- `Mode2RansEncoderTrace.ec`: actual 내부 loop가 사용하는 유한 phase, byte prepend, segment, prefix frame, no-wrap 보조정리(lemma)를 제공한다.
- `Mode2RansEncoderActualInner.ec`: 실제 generated `_rans_encode` 내부 loop에 위 불변식(invariant)을 설치하고, 별도 실제 Hoare 정리(theorem)로 성공 offset과 size 범위(range)를 산출한다.

주의 5.3 (Week 10의 국소 **PROVED**와 당시 부모 **PARTIAL**). Week 10 기준선에서 위 파일들은 보조정리(lemma)와 국소 실제 부분정확성(actual partial correctness)만 인증했다. 당시에는 전체 actual suffix와 순수 trace_bytes의 등식(equality)이 없어 **OBL-RANS-ENCODE-REFINEMENT**가 **PARTIAL**이었다. 바로 그 부모 간선(edge)을 Week 11의 다음 절이 닫는다. 성공 분기(success branch)의 도달가능성(reachability)과 종료성(termination)은 여전히 인증하지 않는다.

5.7 Week 11에서 닫힌 actual encoder 불변식(invariant)

실제 reverse encoder의 바깥 루프 index를 i , 현재 word state를 x , 뒤에서 감소하는 buffer cursor를 off , buffer를 enc 라 하자. 순수 trace

$$T_i = \text{EncodeTrace}(\text{SymbolSuffix}(\text{symbols}_0, i)) = (x_i, b_i)$$

를 두면 성공 가지의 live 불변식(live invariant)은 다음 다섯 조건이다 [6].

$$\begin{aligned} 0 \leq i \leq 1024, \quad x = x_i, \quad off + |b_i| = 1024, \\ \text{SegmentMatches}(enc, off, b_i), \quad \text{PrefixFrame}(enc_0, enc, off), \quad 4 \leq off. \end{aligned}$$

Week 11에는 7개 타깃이 더해졌다. tail-aware inner 불변식(invariant)은 최초 배열 enc_0 와 $\text{EncodeTrace}(tail).bytes$ 를 actual byte write 전후에 보존하고, 실제 guard 종료에서 정확한 0.2 정규화 길이(normalization length)를 회복한다. generated word-step 보조정리(lemma)는 실제 table load와 protect erase 뒤의 word update를 순수 fast step과 같게 만들고, 마지막 네 generated store는 직렬화된 상태(serialized state)를 누적 suffix 앞에 붙인다.

그 결과 실제 절차의 성공 사후조건(success postcondition)은

$$\begin{aligned} \text{SegmentMatches}(enc', off', \text{TraceBytes}(\text{SymbolList}(\text{symbols}_0))), \\ off' + |\text{TraceBytes}(\text{SymbolList}(\text{symbols}_0))| = 1024, \\ \text{PrefixFrame}(enc_0, enc', off') \end{aligned}$$

로 컴파일되었다.

검증된 명제(verified proposition) 5.4 (OBL-RANS-ENCODE-REFINEMENT). actual_rans_encode_trace_closure는 실제 generated `_rans_encode`가 반환한다면 실패하거나, 성공 시 위 세 관계(relation), offset/size 범위(range), $bad = 0$ 을 모두 보인다. actual_rans_encode_trace_refinement는 전체 반환 접미부(returned suffix) 등식(equality)을 부모 refinement 표면에 명시적으로 노출한다. 성공은 전제가 아니라 actual 반환값에 따른 결론의 한 가지(branch)다.

주의 5.5 (부분정확성(partial correctness)의 한계). 이 정리(theorem)는 actual encoder의 종료성(termination), 성공 도달가능성(success reachability), all-6 symbol 성공 증인(success witness)을 보이지 않는다. 따라서 **OBL-RANS-ACTUAL-SUCCESS-WITNESS**는 **PARTIAL**이다.

5.8 Week 12에서 닫힌 actual decoder 불변식(invariant)

Week 12는 actual forward decoder index를 i , state를 x , 읽기 cursor를 off_d 라 할 때 다음 불변식(invariant)을 generated outer/inner loop에 직접 설치했다 [11, 5].

1. 이미 출력한 symbol prefix가 $s_0, \dots, s_{i-1}$ 와 같다;

2. 현재 state가 x_i 이고 현재 cursor가 c_i 다;
3. table lookup이 s_i 를 선택하고, normalization segment를 정확히 $[c_i, c_{i+1})$ 에서 읽는다;
4. 출력 배열의 아직 쓰지 않은 tail에 대한 frame이 유지된다;
5. 종료 시 $x = 2^{23}$, $off_d = size$, $bad = 0$ 이다.

역사적 actual_rans_decode_mode2_control은 실제 tables/state fields와 $bad \in \{0, 1\}$ 만 고정했다. 새 입력 술어(input predicate) actual_mode2_decoder_trace_input은 expected symbol array, exact trace_bytes, 모든 normalization read의 pointwise relation, state fields (1024, encoded_size, 13), 실제 generated tables를 한데 묶는다.

검증된 명제(verified proposition) 5.6 (OBL-RANS-DECODE-REFINEMENT). actual_rans_decode_trace_refinement는 실제 generated_rans_decode를 직접 열어 위 exact-trace 입력 아래 모든 1024개 symbol을 복원하고, $off = encoded_size$, $bad = 0$, decoded tail frame을 결론낸다. decoder_outer_trace_exit_components는 내부적으로 $x = 2^{23}$ 을 유도하여 actual final-state reject도 통과시킨다.

이 정리(theorem)는 Hoare 부분정확성(Hoare partial correctness)이므로 decoder 종료성(termination)을 보이지 않는다. exact-trace 입력은 비순환적(non-circular)이며, Week 13은 actual harness가 그 입력을 산출한다는 별도 합성 정리(composition theorem)를 추가했다.

5.9 Week 13에서 닫힌 actual harness core 합성(composition)

Mode2RansActualInverse.ec의 harness는 추상 조립도가 아니라 세 실제 generated 절차를 다음 순서로 호출한다. 같은 파일의 정리(theorem)는 이 호출의 control만 닫고, Week 13의 의미론적 closure는 Mode2RansCoreActualInverse.ec에 따로 있다.

$$\text{RansEncode} \rightarrow \text{CopyEncodedSuffix} \rightarrow \text{RansDecode}.$$

단, encoder가 $bad = 0$ 을 반환한 경우에만 뒤의 두 호출을 실행한다.

기존 harness control 결론은

$$\begin{aligned} \text{encoderBad} &\in \{0, 1\}, \\ \text{decoderRan} &\iff \text{encoderBad} = 0, \\ \text{decoderRan} &\implies (\text{count}, \text{size}, m) = (1024, \text{encodedSize}, 13) \end{aligned}$$

이다. Week 13의 Mode2RansCoreCompositionBridge.ec는 encoder segment_matches와 copy_slice_eq를 copied buffer의 exact trace로 합성하고, 이를 decoder의 pointwise read와 configured state 입력 술어(input predicate)로 운송한다. 특히 modular (W64) 뺄셈을 정수 뺄셈(integer subtraction)으로 몰래 바꾸지 않고 actual success bound에서 no-wrap을 유도한다.

검증된 명제(verified proposition) 5.7 (OBL-RANS-CORE-INVERSE). actual_rans_encode_copy_decode_inverse는 actual encoder 성공을 전제로 받지 않는다. 실패 시 actual copy와 decoder가 실행되지 않고 초기 decoded array/state가 보존된다. 성공 시 다음 의미론적 결론(semantic conclusion)이 같은 harness 정리(theorem)의 사후조건(postcondition)에 들어간다.

$$\text{decoderBad} = 0, \quad \text{off}_{\text{final}} = \text{size}, \quad \text{decoded}[0..1024) = \text{symbols}[0..1024).$$

decoder 하위 증명은 내부 종료 상태(internal exit state) $x = 2^{23}$ 도 유도하지만 harness ABI는 이를 공개하지 않는다. decoded tail [1024, 2048)은 frame으로 보존된다. 이 결과는 성공 조건부 부분정확성(success-conditioned partial correctness)이다.

실제 success branch가 도달 가능하다는 EasyCrypt witness도 아직 없다. pure trace의 concrete witness, canonical all-zero precondition witness 또는 decoder state의 size-4 witness는 concrete all-6 입력에서 actual encoder가 $bad = 0$ 을 반환한다는 success witness를 대신하지 않는다.

5.10 full HBZ wrapper의 올바른 목표 모양

actual encoder는 실패를 표현할 수 있으므로, canonical h 만으로 무조건적인 round trip을 선언해서는 안 된다. 현재 필요한 부모 명제(parent proposition)는 적어도

$$size = 0 \quad \vee \quad (size \neq 0 \wedge bad = 0 \wedge decoded[0..1024) = h[0..1024))$$

처럼 failure branch를 드러내야 한다. nonzero size bounds와 buffer frames도 함께 필요하다.

미해결 의무 5.8 (OBL-SIG-HBZ-ENCODE-DECODE). actual `_encode_hb_z1_full` 과 `_decode_hb_z1_full` 을 직접 열고, 이미 증명된 prepare/apply 역함수(inverse function)와 actual rANS core 정리(theorem)를 wrapper-level size/frame에 맞춰 합성해야 한다. 이 정리(theorem)가 컴파일되기 전에는 두 번째 (h) codec으로 이동하지 않는다.

rANS 층(layer)	상태
13-symbol interval/frequency와 actual table certificate	PROVED
pure quotient/slot/fast-step inverse	PROVED
0/1/2-byte normalization과 W32/W64 leaf semantics	PROVED
canonical <i>xs/cuts/bytes</i> trace와 state bounds	PROVED
actual array/list suffix와 word-step bridge	PROVED
actual 네-byte 최종 serialization 합성	PROVED
actual encoder 성공 offset/size 범위	PROVED (부분정확성 (partial correctness))
actual 내부 normalization byte/frame/no-underflow	PROVED (tail-aware exact 0..2 exit)
actual encoded suffix copy와 tail frame	PROVED
actual encoder-copy-decoder branch control	PROVED (control 만)
actual encoder가 pure trace 전체를 산출함	PROVED (성공 조건부 부분정확성 (success-conditioned partial correctness))
actual decoder가 exact trace를 소비함	PROVED (1024-symbol 복원, exact consumption, tail frame)
actual encoder-copy-decoder core inverse	PROVED (성공 조건부 부분정확성 (success-conditioned partial correctness))
actual encoder success witness	PARTIAL (success reachability와 termination 부재)
full HBZ encode/decode	PARTIAL (production wrapper 합성 미완)

6 열린 경계와 과장 금지선

6.1 가장 좁지만 중요한 의미론(semantics) 간극(gap): product/replay

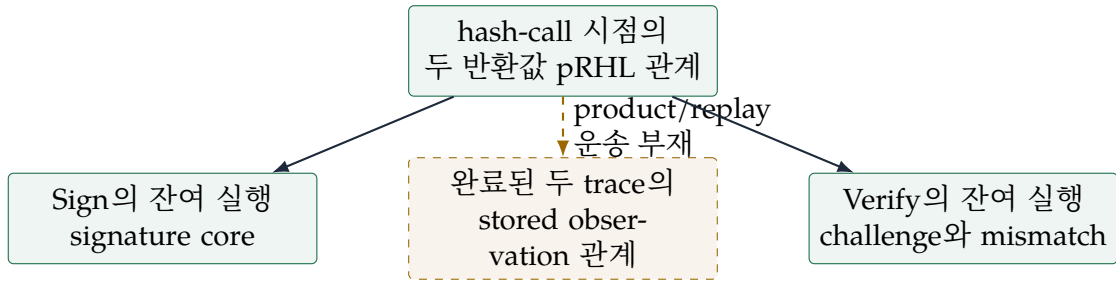
현재 raw μ 선에는 다음 두 사실이 각각 존재한다.

1. 실제 generated Sign/Verify hash 호출의 반환값은 관련 입력에서 같은 32바이트 접두부를 갖는다;
2. 실제 Sign 후 Verify 전체 실행은 ghost trace와 exact하게 대응하고, trace는 각 hash 결과를 `observed_mu`에 저장한다.

원하는 결론은 accepted Verify 사후상태(post-state)에서

$$\text{VerifyResult} = 0 \implies \text{MuPrefix}_{32}(\text{SignTrace.observed_mu}, \text{VerifyTrace.observed_mu})$$

이다. 수학적으로는 1과 2를 합치면 자명해 보이지만, 현재 program-logic 표면에서는 두 명제(proposition)의 변수(variable)가 같은 시점에 존재하지 않는다.



hash-call의 pRHL goal에는 $res\{1\}, res\{2\}$ 가 있지만 전체 trace가 끝난 unary post-state에는 이 지역 반환변수가 사라진다. 반대로 hash-call 지점에서 pRHL 정리(theorem)를 적용하면 Sign과 Verify의 서로 다른 잔여 continuation을 처리해야 한다. Sign 쪽 continuation을 일방적으로 버리려면 아직 없는 losslessness 또는 적절한 product rule이 필요하다.

미해결 의무 6.1 (OBL-MU-PRODUCT-REPLAY). 반환값 수준의 region-local pRHL 정리(theorem)를 이미 완료된 sequential trace의 stored observation 관계(relation)로 운반하는 sound rule 또는 증명 구조가 필요하다.

이 의무는 Week 7에 **DEFERRED** / explicit paper boundary로 동결되었다 [13]. 다음 접근은 의도적으로 거부되었다.

- observation equality를 전제로 가정하여 결론을 재진술하기;
- SHAKE locality axiom을 새로 추가하기;
- 또 다른 caller/observation wrapper로 theorem surface를 계속 넓히기;
- termination 근거 없이 Sign continuation을 버리기.

따라서 $\delta_{\mu, \text{raw_api_accept}} = 0$ 은 현재 주장하지 않는다. 이것은 byte-locality 정리(theorem)가 부족해서가 아니라 프로그램 논리의 운송 화살표가 없어서 생긴 의미론 간극이다.

6.2 accepted-path와 legitimate-signature correctness는 다른 방향이다

실제 Verify에 대해 증명된 방향은

$$\text{Accept} \implies \text{TailReached} \implies \text{HashInputsBound}$$

이다. 이는 이미 accept한 실행을 분석하는 security-facing lane이다.

기능정확성에 필요한 방향은

$$\sigma \leftarrow \text{Sign}(sk, M) \Rightarrow \text{Verify}(vk, M, \sigma) = \text{Accept}$$

이다. 이를 위해서는 Sign이 만든 서명이 unpack/norm/hint/challenge gate를 모두 통과한다는 사실이 필요하다. 첫 방향의 정리(theorem)를 둘째 방향으로 뒤집을 수 없다.

미해결 의무 6.2 (OBL-SIGN-OUTPUT-TAIL-REACH). 실제 Sign output이 실제 Verify의 early-reject tree를 통과해 tail에 도달함을 증명해야 한다. 현재 signature prefix inverse만 닫혔고 full suffix, norm, hint, arithmetic correctness가 남아 있다.

OBL-SIGN-VERIFY-CORRECTNESS는 이 의무와 나머지 산술·challenge 정확성에 막혀 **BLOCKED**다.

6.3 full signature codec에서 남은 표현론

prefix의 단면-수축(section-retraction)을 full codec으로 확장하려면 적어도 다음 사각형(square)을 닫아야 한다.

- 두 size-delta metadata byte가 pack/unpack에서 같은 크기를 나타냄;
- HBZ rANS actual full wrapper가 성공 조건 아래 역함수(inverse function)임;
- 두 번째 h suffix codec의 encode/decode;
- trailing zero padding의 정준성(canonicity)과 parser의 zero gate;
- packer/unpacker가 목표 밖 배열 tail을 보존함;
- 실제 Sign이 canonical 입력과 성공 buffer bound를 산출함.

따라서 **OBL-SIG-PREFIX-CODEC**은 제한된 의미에서 **PROVED**지만, **OBL-SIG-FULL-CODEC-MODE2**는 **BLOCKED**다. prefix inverse와 full inverse를 같은 명제(proposition)로 부르면 안 된다.

6.4 대수(algebra)·수치(numerics)·확률(probability) 층(layer)에 남은 의무

현재 활발한 전선은 production full-HBZ wrapper지만, 전체 구현 refinement에는 다음 독립적인 의무들이 남는다.

public-key algebra/NTT

R_q -모듈(module) 계산, NTT 표현, public-key 관계가 실제 구현과 일치함.

sampler distribution uniform/small/hyperball sampler와 실제 random tape의 출력분포(output distribution)가 논문 분포(distribution)와 일치하거나 명시적 거리 상계(distance bound)를 가짐.

rejection과 종료 KeyGen/Sign retry가 종료하고, 수락 분포(accepted distribution)와 retry loss가 계산됨.

highbits/LSB 실제 coefficient decomposition, rounding, packing이 논문의 정수 분해(integer decomposition)와 일치하며 challenge 양쪽 입력이 같음.

FIPS202 transcript 모든 domain separator, 길이, absorb 순서, squeeze 길이가 고정 명세와 byte-for-byte 일치함.

FFT tie policy fixed-point FFT와 경계 동률 처리까지 analytic predicate와 연결됨.

two-transcript extraction

실제 byte-level challenge와 accepted signature instance를 paper/ROM reduction의 두 transcript 추출에 연결함.

ImplementationSecurityGoals.ec는 이 항목들을 `obl_* operation`으로 열거한다. 이 파일이 컴파일된다는 것은 목표의 이름과 합성 인터페이스가 well-formed하다는 뜻이지 각 boolean이 참임을 증명했다는 뜻이 아니다.

6.5 최상위 기능정확성은 아직 명세다

TopLevelGoals.ec는

$$\begin{aligned} \text{FC}_{\text{KG}} : & \quad \text{JKeyGen}_2 = \text{PaperKeyGen}_2, \\ \text{FC}_{\text{Sign}} : & \quad \forall sk, M, \text{JSig}_2(sk, M) = \text{PaperSig}_2(sk, M), \\ \text{FC}_{\text{Verify}} : & \quad \forall vk, M, \sigma, \text{JVerify}_2(vk, M, \sigma) = \text{PaperVerify}_2(vk, M, \sigma) \end{aligned}$$

를 boolean specification으로 둔다. 여기서 Sign/KeyGen의 등식(equality)은 확률분포(probability distribution)의 등식(equality in distribution)을 포함한다. 현재의 prefix, memory, transcript, codec 정리(theorem)는 이 목표에 필요한 결정론적 하위 사각형이지만 전체 등식 자체는 아니다.

6.6 보안 합성식은 아직 조건부 인터페이스다

구현과 논문 스킴 사이의 목표는

$$\text{Adv}_{\text{Jasmin}}^{\text{euf-cma}} \leq \text{Adv}_{\text{paper}}^{\text{euf-cma}} + \delta_{\text{KeyGen}} + q_{\text{Sign}} \delta_{\text{Sign}} + \delta_{\text{Verify}} + \delta_{\text{Encoding}}$$

의 모양이다. 현재 파일은 손실 변수(loss variable)를 선언하고 부등식(inequality)의 interface를 정의하지만, 각 δ 를 모두 달아 최종 정리(theorem)로 합성하지 않았다.

논문 수준 reduction interface는 다시 MLWE, BST-MSIS, ROM 및 rejection/fork 오차로 내려간다. SecurityAssumptions.ec는 최종적으로 남길 수 있는 암호학적 인터페이스를 MLWE, BST-MSIS, ROM 세 종류로 제한한다. 하지만 구체적 게임 instance와 매개변수화된 reduction이 구현의 정확한 byte-level scheme에 연결되었다는 뜻은 아니다.

주의 6.3 (선언 종류의 구분). 연산(operation), 가정(assumption), 정리(theorem)는 서로 다르다. `opobl_sampler_distribution:bool`. 처럼 본문 없이 선언된 operation은 목표를 표현하는 추상 기호(abstract symbol)다. 그것을 참이라고 가정하는 axiom과 동일하지 않고, 그것을 참이라고 증명한 theorem도 아니다. 프로젝트의 검증 스크립트는 authored axiom과 proof hole을 별도로 금지·검색한다.

6.7 신뢰 경계와 범위 밖 항목

현재 증거 사슬은 다음을 신뢰하거나 고정한다.

- SHA-256가 기록된 46쪽 HAETAЕ 명세 PDF [1];
- pinned haetae-ref-jasmin/ 구현과 focused extraction [2];
- EasyCrypt kernel, Jasmin model, 호출된 SMT prover와 build toolchain;
- source hash와 generated extraction/certificate manifest.

다음은 이 sprint의 결론에 포함되지 않는다.

- mode 2 이외의 parameter set;
- 물리적 side channel, fault, RNG 품질;
- memory safety나 constant-time이 EUF-CMA에서 자동으로 따라온다는 주장;
- 실제 하드웨어/컴파일러 전체의 end-to-end correctness;
- full KeyGen, Sign, Verify 또는 구현 EUF-CMA가 완료되었다는 주장.

6.8 현재 주장 행렬

주장	상태	정확한 경계
packed SK의 VK prefix	PROVED	실제 generated packer/Internal KeyGen의 반환 시 첫 992바이트
raw μ generated seam	PROVED	raw branch, 관련 key/pre/message region에서 64-to-32 prefix
position-64 challenge suffix	PROVED	동일 초기 state/position에서 32-byte absorb
actual API key copy/address/frame	PROVED	각 명명된 valid/canonical/disjoint 계약 아래
accepted Verify tail/input binding	PROVED	accept $\Rightarrow$ tail/hash inputs 방향
stored observed μ equality	PARTIAL	product/replay transport 부재
signature prefix inverse	PROVED	canonical challenge/low, 앞 1056바이트
HBZ leaf inverse	PROVED	앞 1024 coefficient, rANS 제외
actual rANS table와 pure step	PROVED	concrete 13-symbol table, 명시된 state range
pure normalization byte trace	PROVED	canonical symbol list, actual loop 아님
actual array/list · word-step bridge	PROVED	1024-symbol suffix 점화식(recurrence), literal table word 등식(equality)
actual final serialization leaf	PROVED	네 번의 실제 store와 누적 trace의 little-endian 직렬화(serialization) 합성
actual encoder success size	PROVED	반환한다면 실패이거나 $0 \leq off \leq 1020$, $4 \leq 1024 - off \leq 1024$
actual inner normalization	PROVED	byte segment · frame · no-underflow와 정확한 순수 0..2 exit
actual encoder trace refinement	PROVED	성공 조건부 부분정확성 (success-conditioned partial correctness)의 전체 suffix 등식(equality)
actual decoder trace refinement	PROVED	exact-trace 부분정확성(partial correctness), 1024-symbol 등식(equality), 정확한 byte 소비와 tail frame
actual rANS harness control	PROVED	실제 호출 순서와 성공/실패 분기만 완료

주장	상태	정확한 경계
actual rANS core inverse	PROVED	actual encoder/copy/decoder의 성공 조건부 부분정확성(success-conditioned partial correctness)
actual encoder success witness	PARTIAL	concrete all-6 입력의 $bad = 0$ 도달성과 encoder/decoder 종료성 부재
production full-HBZ wrapper inverse	PARTIAL	prepare/apply와 core는 증명됨; success-conditioned wrapper size/frame 합성 미완
full signature codec	BLOCKED	HBZ wrapper 뒤에도 h , metadata, padding과 residual frame 잔여
Sign output tail reach	PARTIAL	prefix만 닫힘; norm/hint/arithmetic 잔여
FC-KeyGen/Sign/Verify	SPECIFIED/PARTIAL	하위 결정론적 seam만 일부 증명
구현 EUF-CMA	SPECIFIED	loss와 paper reduction 연결 미완

7 의존성에 따른 다음 작업과 참여 지점

7.1 현재 단일 우선순위

Week 13 종료의 판정은 GO-CORE다. 첫 의무는

actual_encode_hb_z1_full과 _decode_hb_z1_full을 prepare/apply 역함수(inverse function) 및 증명된 rANS core와 합성하는 성공 조건부 full-HBZ wrapper 정리(theorem)

이다. GO-CORE는 actual encoder, suffix copy, exact-trace decoder가 한 harness의 성공 조건부 core 역함수(inverse function)로 컴파일되었다는 뜻이다. actual 성공 도달성(success reachability), 종료성(termination), production full-HBZ wrapper가 증명되었다는 뜻은 아니다. 위 wrapper 합성(composition)이 컴파일되기 전에는 h codec으로 이동하지 않는다.

7.2 1단계 완료: encoder

OBL-RANS-ENCODE-REFINEMENT는 실제 _rans_encode를 여는 actual_rans_encode_trace_closure와 그 확장된 따름정리(corollary) actual_rans_encode_trace_refinement로 닫혔다. 성공 분기(success branch)의 사후조건(postcondition)은 다음을 동시에 준다.

- actual output suffix가 canonical symbol list의 trace_bytes와 점별로 같음;
- 바깥 반복(outer iteration)마다 actual word state가 suffix의 순수 encode_trace state와 같음;
- actual off 와 trace byte 길이가 $off + |TraceBytes| = 1024$ 를 만족함;
- 이미 증명된 final 4-byte state 직렬화(serialization)를 normalization suffix 앞에 합성하여 segment 순서를 확정함;
- 쓰지 않은 접두부(unwritten prefix)의 frame을 유지함;
- 실패 branch를 숨기지 않음.

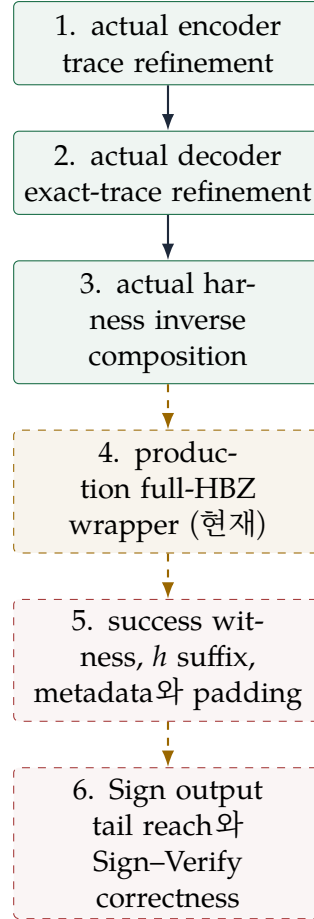


그림 4: codec 기능정확성 선의 권장 순서. Week 11, Week 12, Week 13에 앞의 세 간선(edges)이 차례로 닫혔다.

Week 10의 array/list suffix 점화식(recurrence), actual reciprocal word step, 내부 byte/frame/no-underflow, final serialization leaf와 성공 offset/size 범위는 Week 11의 tail-aware 바깥/안쪽 불변식(outer/inner invariant), generated table load-protect-word-update 보조정리(lemma), final-store 합성(composition)에 연결되었다. 따라서 임의의 valid trace나 새 증인 버퍼(witness buffer)가 아니라 실제 반환 버퍼의 suffix가 정준 trace 바이트(canonical trace bytes)와 일치한다.

다만 이는 Hoare 부분정당성(Hoare partial correctness)이다. 실제 실행이 성공 분기(success branch)에 도달한다는 성공 증인(success witness)과 종료성(termination)은 **OBL-RANS-ACTUAL-SUCCESS-WITNESS**에 남아 있다.

7.3 2단계 완료: decoder

OBL-RANS-DECODE-REFINEMENT는 실제 `_rans_decode`를 여는 `actual_rans_decode_trace_refinement`로 닫혔다. exact-trace 입력 관계(input relation) 아래 다음이 귀납법(induction)으로 증명되었다.

- iteration i 에서 decoded prefix가 원래 symbol prefix와 같음;
- state/cursor가 pure x_i, c_i 와 같음;
- actual table lookup이 구간 분할(interval partition)의 유일한 s_i 를 선택함;
- nested normalization loop가 β_i 를 정확한 순서로 소비함;
- 내부 loop 종료에서 $state = 2^{23}$ 을 유도하고, 반환값에서 $off = size, bad = 0$ 을 결론냄;

- decoded tail frame.

encoder와 decoder를 직접 lockstep으로 묶는 방법은 reverse/forward 방향과 중간 buffer copy 때문에 proof surface가 커진다. 이미 구성된 순수 trace를 공통 매개체로 삼아 두 refinement를 독립적으로 검증했다. 단, 이 decoder 정리(theorem)는 입력 trace를 전제로 받는 Hoare 부분정당성(Hoare partial correctness)이며 종료성(termination)을 주지 않는다.

7.4 3단계 완료: actual harness inverse

OBL-RANS-CORE-INVERSE 는 `Mode2RansCoreActualInverse.ec` 의 `actual_rans_encode_copy_decode_inverse`로 닫혔다. 이 정리(theorem)는 `Mode2RansActualInverse.ec`에 정의된 actual harness를 직접 대상으로 삼아 다음 의미론적 결론을 낸다. 두 파일명은 각각 “harness 정의/control”과 “Week 13 semantic inverse”를 가리키며, 후자는 전자의 control-only 정리(theorem)를 의미론적 결과로 잘못 읽지 않도록 별도 증명된 것이다.

- encoder 실패 분기(failure branch)를 숨기지 않고 decoder를 실행하지 않음;
- encoder 성공 suffix의 `segment_matches`를 actual copy의 `slice_eq`에 함성함;
- copy된 buffer에서 decoder의 pointwise exact-read 전제를 산출함;
- actual decoder 반환에서 $bad = 0$, exact consumption, 1024-symbol 등식(equality), tail frame을 얻음;
- 임의의 trace, 새 증인 버퍼(witness buffer), encoder 성공을 외부 전제로 넣지 않음.

이 정리(theorem)는 성공 조건부 부분정확성(success-conditioned partial correctness)이다. canonical all-zero array는 전제의 비공허성(non-vacuity)을 보이지만, encoder 성공의 존재나 전체 실행의 종료성(termination)을 자동으로 주지 않는다. 성공 도달성은 **OBL-RANS-ACTUAL-SUCCESS-WITNESS**의 별도 의무다.

7.5 4단계 현재 완료 기준: production full-HBZ wrapper

OBL-SIG-HBZ-ENCODE-DECODE를 닫으려면 `actual_encode_hb_z1_full`과 `_decode_hb_z1_full`을 직접 여는 정리(theorem)가 적어도 다음을 결론내야 한다.

- canonical HBZ coefficient를 actual prepare가 정준 기호열(canonical symbol stream)로 옮김;
- actual rANS core 정리(theorem)의 실패/성공 분기를 그대로 보존함;
- 성공 시 actual apply가 복원된 symbol을 원래 coefficient로 되돌림;
- wrapper-level size, state, 입력/출력 tail frame을 연결함;
- actual success witness나 decoder 결과를 외부 전제로 다시 요구하지 않음.

이 단계 역시 우선 성공 조건부 부분정확성(success-conditioned partial correctness)으로 닫을 수 있다. 실제 성공 가지의 도달가능성(reachability)과 종료성(termination)은 별도 정리(theorem)다.

7.6 5-6단계: 작은 부모부터 닫기

production full-HBZ wrapper 함성(composition)이 컴파일되면 다음 순서가 자연스럽다.

1. 적어도 하나의 canonical input에 대해 actual encoder success witness를 구성해 성공 branch의 non-vacuity를 보인다;

2. 두 번째 h suffix codec을 같은 leaf-table-loop-wrapper 층으로 분석한다;
3. metadata와 padding을 연결해 full signature pack/unpack inverse를 얻는다;
4. Sign의 실제 출력이 canonical codec · norm · hint · arithmetic 전제를 산출함을 보이고 Verify tail reach로 연결한다.

각 부모를 닫을 때 자식 정리(child theorem)의 전제가 실제 앞 절차의 결론에서 나오는지를 확인해야 한다. external precondition으로 다시 요구하면 합성이 아니라 조건부 재진술이 된다.

7.7 동결된 product/replay 선은 별도로 다룬다

codec 선과 독립적으로 **OBL-MU-PRODUCT-REPLAY**는 explicit paper boundary로 남아 있다. 재개하려면 wrapper 추가가 아니라 다음 중 하나가 필요하다.

- hash 반환값을 ghost observation에 저장하는 시점까지 보존하는 sound relational sequencing lemma;
- 두 continuation을 함께 처리하는 product program과 그 semantics;
- Sign continuation의 losslessness/termination을 먼저 증명한 뒤 사용할 수 있는 정당한 one-sided elimination;
- hash-time snapshot을 trace가 보존하도록 하는 최소 instrumentation 변경과 그 actual-to-trace exactness 재증명.

어느 선택도 단순한 byte lemma가 아니다. program logic 설계와 theorem surface trade-off를 먼저 검토해야 한다.

7.8 순수 대수학자가 바로 기여할 수 있는 문제

참여 질문 7.1 (rANS를 제한된 전단사(restricted bijection)로 패키징하기). 집합(set)

$$\mathcal{X}_s = \{x : 2^{23} \leq x < 2^{31}\}, \quad \mathcal{N}_s = \{r : 1 \leq r < x_{\max}(s)\}$$

와 0/1/2-byte 섬유(fiber)를 사용해 normalization-fast-step을 명시적인 부분 전단사(partial bijection)로 패키징할 수 있는가? 현재 개별 lemma를 actual loop invariant에서 재사용하기 쉬운 하나의 귀납 정리(induction theorem)로 묶는 것이 목표다.

참여 질문 7.2 (표현의 몫(quotient)과 정준 단면(canonical section)). signature 각 구성요소에 대해 raw word 공간(word space), 바이트 몫(byte quotient), 정준 대표자계(canonical system of representatives), decoder 단면(section)을 한 번에 기술하는 공통 구조를 만들 수 있는가? 새 추상화는 실제 기존 술어를 재사용하고, prefix/HBZ/향후 h codec의 중복만 제거해야 한다.

참여 질문 7.3 (NTT와 R_q 표현). 기존 NTT · KeyGen 결과의 정확한 명제(proposition)/전제(premise) 경계를 유지하면서, 계수 표현(coefficient representation)과 NTT 표현(NTT representation) 사이의 가환사각형(commutative square)을 최상위 obl_public_key_algebra_ntt에 어떻게 연결할 수 있는가? 종이 위 CRT 동형사상(CRT isomorphism)만 인용하지 말고 실제 procedure theorem이 필요하다.

참여 질문 7.4 (high/low decomposition의 정수 경계(integer bound)). 논문의 $r = \alpha \text{ HighBits}(r) + \text{LowBits}(r)$ 와 실제 고정폭 rounding/sign-extension/packing 사이의 정준 대표자(canonical representative) 조건을 정확히 분리할 수 있는가? 이 작업은 전체 challenge input과 norm/hint correctness의 대수적 중심이다.

참여 질문 7.5 (rejection distribution). partial map으로 모델링된 retry loop를 확률 커널(probability kernel)로 올리고, 수락 분포(accepted distribution)와 termination/loss bound를 논문 sampler에 연결할 수 있는가? 결정론적 Hoare 정리(deterministic Hoare theorem)와 분포 정리(distribution theorem)를 섞지 않는 interface 설계가 필요하다.

7.9 대수기하학자의 기술이 유용한 곳

현재 작업이 스킴 이론적(scheme-theoretic) 증명은 아니지만 다음 습관은 직접 유용하다.

- 전체 객체(object) 대신 필요한 제한(restriction)만 비교하는 국소-대역(local-to-global) 사고;
- 서로 다른 표현 좌표(chart) 사이의 전이사상(transition map)과 호환성(compatibility) 도식을 명시하는 습관;
- 몫(quotient)과 대표 단면(representative section)을 분리하는 습관;
- 섬유(fiber)가 비어 있지 않음을 witness로 확인하고, 조건부 정리(conditional theorem)의 non-vacuity를 점검하는 습관;
- 큰 목표를 가환사각형(commutative square)과 장애물(obstruction)로 분해하는 습관.

단, 이 습관을 “현재 코드가 층(sheaf)을 형식화한다”는 주장으로 바꾸면 안 된다. 실제 proof term은 배열, word, 메모리, procedure semantics 위에 있다.

7.10 새 정리(theorem)를 제출할 때의 판정 체크리스트

1. 정리(theorem)의 actual/generated/pure 경계를 이름과 주석에 명시했는가?
2. 부모 결론을 전제로 다시 가정하지 않았는가?
3. termination, partial correctness, losslessness를 구분했는가?
4. canonicity, address range, no-wrap, separation 전제가 모두 보이는가?
5. 반환값, global state, ghost observation 중 무엇을 비교하는지 명시했는가?
6. 사용하지 않는 배열·메모리 구간의 frame이 필요한가?
7. 전제의 concrete witness 또는 실제 predecessor가 있어 non-vacuous한가?
8. authored axiom, admit/sorry/abort, debug declaration이 없는가?
9. manifest에 들어가 fresh -no-eco compile되는가?
10. claim ledger와 theorem graph의 부모 상태를 과장 없이 갱신했는가?

이 체크리스트를 통과해야 “수학적으로 그럴듯함”이 “현재 구현에 대해 검증됨”으로 바뀐다.

A 정리(theorem)와 소스 인덱스

이 부록은 본문의 수학 문장을 실제 저장소 선언으로 되돌아가기 위한 지도다. 줄 번호는 편집 때 바뀌므로 파일 경로와 정리 식별자(theorem identifier)를 기준으로 한다. 최종 판정은 항상 해당 EasyCrypt 선언의 전제와 최신 CLAIM_LEDGER.md를 함께 읽어야 한다.

A.1 상태와 범위 문서

경로	역할
PROJECT_SCOPE.md	mode 2 범위, 고정 명세, trust boundary, 완료로 주장하지 않는 항목

경로	역할
CLAIM_LEDGER.md	claim ID별 운영상 source of truth; status, 실제 Jasmin target, EasyCrypt evidence, 남은 전제
THEOREM_GRAPH.md	최상위 보안 목표에서 leaf 정리(theorem)까지의 의존성 DAG
ASSUMPTIONS.md	최종 가정, 구현 계약, non-vacuity, 과장 금지 정책
PRODUCT_REPLAY_DECISION.md	stored μ product/replay gap의 의미론 분석과 동결 결정
SIGNATURE_CODEC_OBLIGATIONS.md	mode 2 signature layout와 staged codec 의무
RANS_PROOF_DESIGN.md	rANS 증명 분해와 설계 선택
RANS_BYTE_STACK_INVARIANT.md	reverse encoder와 forward decoder를 잇는 <i>xs/cuts/bytes</i> invariant
WEEK9_REPORT.md	순수 byte stack과 actual control 경계의 역사적 Week 9 기준선
RANS_ENCODER_INVARIANT.md	actual encoder 바깥 루프 불변식(outer-loop invariant), Week 10 설계 경계와 Week 11 actual trace closure
WEEK10_REPORT.md	역사적 CONTINUE-ENCODER 판정과 50-target 검증 결과
WEEK11_REPORT.md	역사적 GO-ENCODER 판정과 57-target 검증 결과, actual encoder trace closure
RANS_DECODER_INVARIANT.md	actual decoder의 순수 state/cursor/segment 좌표(coordinates), outer/inner 불변식(invariants), exact-trace 경계
WEEK12_REPORT.md	역사적 GO-DECODER 판정, 65-target 검증 결과와 actual decoder refinement
RANS_CORE_COMPOSITION.md	encoder segment, actual copy slice, decoder pointwise read와 configured state 사이의 Week 13 운송 구조
MODE2_HBZ_CODEC_OBLIGATIONS.md	HBZ leaf, rANS core, production wrapper 의무의 최신 층별 상태
WEEK13_REPORT.md	현재 GO-CORE 판정, 67-target 검증 결과, actual core inverse와 full-HBZ wrapper 단일 우선순위

A.2 packed key와 memory bridge

파일	주요 선언	수학적 역할
easycrypt/refinement/keygen/PackedKeyPrefix.ec	vk_prefix_eq_set_after; copied_prefix_step; copied_prefix_complete	접두부 동치관계(equivalence relation)의 순수 배열 invariant
easycrypt/refinement/keygen/ExtractedPackedKeyPrefix.ec	pack_vec_eta_to_prefix_frame; pack_vec2_eta_to_prefix_frame; pack_sk_m23_mode2_vk_prefix	actual packer의 접두부와 이후 쓰기 frame
같은 파일	keypair_full_m23_mode2_return_prefix; keypair_internal_mode2_return_prefix	generated internal KeyGen 반환까지 lift

파일	주요 선언	수학적 역할
easycrypt/support/ Mode2KeyMemoryBridge.ec	keygen_prefix_reaches_mu_ memory	로컬 배열 prefix를 global-memory load 관계로 번역
easycrypt/refinement/ composition/ ApiKeyMemoryBridge.ec	keygen_export_vk_mode2_prefix; keygen_export_sk_mode2_prefix; sign_import_mode2_sk_prefix; verify_import_mode2_vk_prefix	actual copy helper의 export/import 계약
같은 파일	keygen_export_sk_mode2_prefix_ frames_vk; sign_verify_api_ importer_same_inputs	disjoint frame과 Sign/Verify 입력 연결
easycrypt/refinement/ composition/ RawApiAddressBridge.ec	canonical_ui64_address_ roundtrip; mode2_vk_region_bridge; mode2_sk_region_bridge; mode2_sig_region_bridge	canonical integer/W64 주소 대응
easycrypt/refinement/ composition/RawApiKeygen SequentialExport.ec	keypair_raw_api_trace_exports_ matching_prefixes; keypair_raw_ api_exports_matching_prefixes	실제 raw KeyGen caller의 순차 외부 export

A.3 μ , challenge, actual API trace

파일	주요 선언	수학적 역할
easycrypt/refinement/ composition/ ExtractedMuHashPrefix.ec	sign_sk_verify_vk_absorb_mode2; sign_verify_final_squeeze_mu32_ prefix; sign_verify_raw_mu_top_ wrappers_prefix	actual generated helper chain의 key/hash/squeeze 관계
easycrypt/refinement/ composition/ ExactMuTopControl.ec	raw_prelen_shr63_zero; raw_prelen_branch_guard	실제 Verify 분기의 raw path 선택
같은 파일	sf_mu_rawpre_refines_raw_top; verify_hash_mu_raw_refines_ raw_top; sign_verify_generated_ raw_mu_prefix	actual generated top 절차의 64-to-32 μ relation
easycrypt/refinement/ composition/ ExactMode2RawMuComposition. ec	generated_raw_mu_ preconditions_from_keygen_ prefix; generated_raw_mu_to_ challenge_suffix_zero_loss	KeyGen prefix에서 실제 position-64 challenge absorb까지
easycrypt/refinement/ composition/ RegionLocalMuEquivalence.ec	region_eq_head; region_eq_tail; sign_verify_raw_mu_top_ wrappers_prefix_regionwise	whole-memory equality를 read-region equality로 약화
easycrypt/refinement/ composition/ RegionLocalMuTop.ec	sign_verify_generated_raw_mu_ prefix_regionwise	region-local relation을 actual generated top 절차로 lift
easycrypt/refinement/ composition/ RawApiSignOutputFrame.ec	sign_output_copy_exact_and_ frames_reused; sign_raw_api_ frames_reused_regions	actual Sign output과 siglen store 의 frame
easycrypt/refinement/ composition/ RawApiVerifyAcceptTrace.ec	verify_cryptolab_actual_accept_ implies_trace_tail_reached; verify_cryptolab_actual_accept_ binds_hash_inputs	accept $\Rightarrow$ tail과 실제 hash input binding
easycrypt/refinement/ composition/ RawApiDirectObservedMu.ec	raw_sign_then_verify_actual_ exact_trace; sign_then_verify_ trace_accept_implies_tail_reached; sign_then_verify_accept_binds_ observed_inputs	actual sequential Sign-Verify와 ghost trace의 exactness

파일	주요 선언	수학적 역할
같은 파일	직접 stored observed_mu relation 없음	OBL-DIRECT-OBSERVED-MU 가 PARTIAL 인 정확한 위치

A.4 signature prefix와 HBZ leaf

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/Mode2SignaturePrefixCodec.ec	canonical_challenge; canonical_signed_low; prefix_codec_preconditions_satisfiable	정준 대표자(canonical representative) 조건과 non-vacuity
easycrypt/refinement/sign/Mode2SignaturePrefixPack.ec	pack_sig_prefix_mode2_layout	actual pack loop의 challenge/low byte layout
easycrypt/refinement/sign/Mode2SignaturePrefixUnpack.ec	unpack_sig_prefix_mode2_layout	actual unpack loop의 decode layout와 tail frame
easycrypt/refinement/sign/Mode2SignaturePrefixRoundTrip.ec	pack_unpack_sig_prefix_mode2_roundtrip	canonical prefix에서 actual decode-after-encode
easycrypt/refinement/sign/Mode2HbzCodecSpec.ec	canonical_hbz_mode2; prepared_hbz_prefix; decoded_hbz_prefix	HBZ canonical domain과 계수 (coefficient)/기호(symbol) 관계 (relation)
easycrypt/refinement/sign/Mode2HbzPrepare.ec	encode_hb_z1_prepare_mode2_correct	actual $h_i \mapsto h_i + 6$ leaf
easycrypt/refinement/sign/Mode2HbzApply.ec	decode_hb_z1_apply_mode2_correct	actual $s_i \mapsto s_i - 6$ leaf
easycrypt/refinement/sign/Mode2HbzLeafRoundTrip.ec	encode_prepare_decode_apply_mode2_inverse	앞 1024 coefficient의 actual leaf 역함수(inverse function)와 frame
easycrypt/refinement/sign/Mode2HbzActualBoundary.ec	pack_target_encode_hb_z1_full_exact_focused; unpack_target_decode_hb_z1_full_exact_focused	focused extraction과 full signature extraction의 실제 procedure identity

A.5 rANS table, 순수 trace, actual boundary

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/Mode2HbzTableCertificate.ec	hbz_frequency_positive; hbz_frequency_sum; hbz_intervals_cover_scale; mode2_hbz_pack_unpack_tables_identical	13-symbol 분할(partition)과 pack/unpack table 호환성 (compatibility)
같은 파일	hbz_reciprocal_exact; hbz_fast_step_matches_math	actual reciprocal fast quotient의 정수 의미
easycrypt/refinement/sign/Mode2HbzSymbolWordsGenerated.ec	actual_mode2_hbz_tables_certified	generated literal arrays가 certificate를 만족
easycrypt/refinement/sign/Mode2RansCore.ec	pure_rans_step_inverse; hbz_fast_step_decode_inverse; hbz_fast_step_preconditions_satisfiable	순수/fast 단일 단계 역함수 (inverse function)와 witness
easycrypt/refinement/sign/Mode2RansNormalization.ec	encoder_shift8_uint; encoder_low_byte_uint; decoder_append_encoder_byte; encoder_two_byte_decoder_order	actual W32/W64 normalization leaf와 byte 순서

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/Mode2RansByteStack.ec	renorm_bytes_readback; serialize32_parse_inverse; encode_trace_state_bounds; trace_ head_normalization_readback; valid_rans_trace_canonical	canonical <i>xs/cuts/bytes</i> trace와 귀납적(inductive) readback
easycrypt/refinement/sign/Mode2RansSuffixCopy.ec	copy_encoded_suffix_correct	actual suffix slice copy와 output tail frame
easycrypt/refinement/sign/Mode2RansEncodeRefinement.ec	actual_rans_encode_mode2_ control	actual table/count/ <i>bad</i> control의 역사적 경계; Week 11 부모 closure는 아래 표
easycrypt/refinement/sign/Mode2RansDecodeRefinement.ec	actual_rans_decode_mode2_ control; mode2_decode_state_satisfiable	actual decode control과 제한된 state witness의 역사적 경계; Week 12 부모 정리(theorem)는 아래 표
easycrypt/refinement/sign/Mode2RansActualInverse.ec	actual_rans_harness_branches_ on_encoder_result; actual_core_success_result	세 actual 호출의 control 경계; Week 13 semantic inverse는 아래 표

Week 10 actual encoder 타깃. 아래 6개 파일은 모두 authored target manifest에 있고 `-no-eco`로 fresh compile된다. 상태는 각 명명된 명제(proposition)의 범위다. Week 10 당시에는 부모 encoder trace refinement가 아직 없었고, Week 11의 다음 표가 이 국소 결과를 실제 절차(actual procedure)까지 합성한다.

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/Mode2RansArrayListBridge.ec	symbol_suffix_cons; segment_matches_prepend_set; encode_trace_suffix_extension	array/list suffix, segment match, prefix frame, 한 기호 trace 점화식 (recurrence)
easycrypt/refinement/sign/Mode2RansEncoderWordStep.ec	actual_mode2_encoder_word_ step_correct; encoder_word_step_ preconditions_satisfiable	literal reciprocal/shift/bias/ complement actual word 식과 순수 fast step의 등식(equality)
easycrypt/refinement/sign/Mode2RansEncoderInnerProgress.ec	normalization_len_cases; inner_progress_segment_step	순수 0/1/2 normalization state/cursor/byte-segment progress
easycrypt/refinement/sign/Mode2RansEncoderSerialization.ec	actual_w32_le_bytes_serialize; actual_encoder_final_state_ serialization; actual_store_w32_le_prefix_frame	최종 W32 little-endian 네 store 와 frame의 leaf 명제(leaf proposition)
easycrypt/refinement/sign/Mode2RansEncoderTrace.ec	encoder_inner_phase_step; encoder_inner_segment_step	actual inner loop에서 재사용하는 finite phase, byte prepend, segment/frame 보존
easycrypt/refinement/sign/Mode2RansEncoderActualInner.ec	actual_rans_encode_inner_no_ underflow; actual_rans_encode_ success_size_bound	generated <code>_rans_encode</code> 의 실제 nested-loop invariant와 성공 offset/size 부분정확성 (partial correctness)

Week 11 actual encoder closure 타깃. 아래 7개 파일 역시 authored target manifest에 있고 `-no-eco`로 fresh compile된다. 앞 표의 운송 보조정리(transport lemmas)를 tail-aware 불변식(invariant), generated word update, final serialization, 실제 `_rans_encode`의 성공 사후조건(success postcondition)으로 결합한다.

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/Mode2RansEncoderGeneratedWordStep.ec	generated_loaded_nested_word_update; generated_outer_word_update_matches	generated table load와 nested word 식을 순수 fast step에 잇는 보조정리(lemmas)
easycrypt/refinement/sign/Mode2RansEncoderSerializationComposition.ec	actual_store_w32_le_prepend_trace; actual_store_w32_le_prepend_trace_bytes	최종 상태(state)의 네 바이트(byte)를 normalization trace 앞에 합성
easycrypt/refinement/sign/Mode2RansEncoderTailInvariant.ec	encoder_inner_tail_exit_exact; encoder_outer_tail_advance_success	0/1/2-byte 안쪽 종료와 한 기호(symbol) 바깥 전이를 보존하는 tail-aware 불변식(invariant)
easycrypt/refinement/sign/Mode2RansEncoderFinalization.ec	encoder_outer_tail_finalize_success	바깥 반복(outer iteration) 종료 상태에서 final-store trace를 구성
easycrypt/refinement/sign/Mode2RansEncoderGeneratedFinalization.ec	generated_encoder_outer_finalize_success	generated offset 계산과 final-store를 순수 trace 결론에 연결
easycrypt/refinement/sign/Mode2RansEncoderActualTraceClosure.ec	encoder_generated_success_outer_post; actual_rans_encode_trace_closure	실제 _rans_encode의 성공 suffix와 정준 trace의 정확한 등식(equality)
easycrypt/refinement/sign/Mode2RansEncoderOuterRefinement.ec	actual_rans_encode_trace_refinement	성공 offset/size/frame을 펼쳐 제시하는 성공 조건부 부분정당성(success-conditioned partial correctness) 따름정리(corollary)

Week 12 actual decoder semantic-refinement 타킷. 아래 8개 파일도 authored target manifest에서 -no-eco로 fresh compile된다. exact trace의 순수 좌표(coordinates), 구체적 table/word 산술(arithmetic), 0/1/2-byte replay, actual outer/inner loop, generated top Hoare 정리(theorem)를 층별로 분리한다.

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/Mode2RansDecoderCursor.ec	decoder_state_at; decoder_cursor; decoder_parse32_from_trace	순수 trace의 state/cursor/segment 좌표(coordinates)와 초기 LE32 parse
easycrypt/refinement/sign/Mode2RansDecoderWordStep.ec	actual_mode2_decoder_lookup_symbol; actual_mode2_decoder_step_correct	concrete symbol_words lookup과 수학적 decoder step
easycrypt/refinement/sign/Mode2RansDecoderNormalization.ec	decoder_replay_complete; decoder_replay_guard_exact; decoder_replay_word_step	normalization segment의 exact 0/1/2-byte replay와 guard
easycrypt/refinement/sign/Mode2RansDecoderActualWord.ec	actual_mode2_decoder_word_update_correct	actual W32/W64 word update와 hbx_math_decode_step의 등식(equality)
easycrypt/refinement/sign/Mode2RansDecoderGeneratedStep.ec	generated_decoder_symbol_expected; generated_decoder_word_update_matches	generated table loads의 expected symbol 선택과 word-step 운송(transport)
easycrypt/refinement/sign/Mode2RansDecoderCursorSteps.ec	decoder_cursor_step; decoder_cursor_final	pure trace cut의 한 단계 점화식(recurrence)과 exact final consumption
easycrypt/refinement/sign/Mode2RansDecoderActualTrace.ec	actual_mode2_decoder_trace_input; decoder_outer_trace_live; decoder_outer_trace_exit_components	actual outer/inner 불변식(invariants), $x = 2^{23}$, exact cursor와 output frame
easycrypt/refinement/sign/Mode2RansDecoderTopHoare.ec	actual_rans_decode_trace_refinement	actual generated _rans_decode의 exact-trace 부분정확성(partial correctness)

Week 13 actual core composition 타킷. 아래 2개 파일은 Week 11 encoder 정리(theorem), Week 12 decoder 정리(theorem), actual copy 정리(theorem)를 한 unary harness에 합성하며 -no-eco로 fresh compile된다.

파일	주요 선언	수학적 역할
easycrypt/refinement/sign/ Mode2RansCore CompositionBridge.ec	copied_suffix_is_exact_trace; trace_bytes_trace_segment_nth; segment_matches_implies_exact_ decoder_segment_input; segment_matches_implies_ decoder_word_reads	encoder suffix와 actual copy를 global trace, local segment, decoder pointwise/W64 read 전제로 운송하는 비순환적 bridge 보조정리(bridge lemmas)
같은 파일	actual_decoder_input_from_ copied_trace; configured_decoder_state_fields; actual_decoder_input_from_ configured_trace; encoder_ success_size_word_bridge; core_composition_preconditions_ satisfiable	copied trace, configured state와 no-wrap size를 Week 12 decoder 입력 술어로 묶고 canonical all-zero 입력으로 semantic precondition의 일관성만 확인
easycrypt/refinement/sign/ Mode2RansCore ActualInverse.ec	actual_rans_encode_copy_ decode_inverse	세 actual generated 절차의 실패/성공 합(disjunction)과 1024-symbol round trip을 한 harness에서 보이는 성공 조건부 부분정확성(success-conditioned partial correctness)

주의 A.1 (닫힌 core 정리(theorem)와 남은 production 경계). actual encoder 성공 suffix, actual suffix copy, actual decoder exact trace는 actual_rans_encode_copy_decode_inverse 안에서 합성되었다. 그러나 이 Hoare 정리(theorem)는 실제 성공 증인(success witness), encoder/decoder 종료성(termination), production full-HBZ wrapper 역함수(inverse function)를 주지 않는다.

A.6 최상위 목표와 가정 표면

파일	읽는 법
easycrypt/specs/ TopLevelGoals.ec	FC_KeyGen, FC_Sign, FC_Verify, ImplementationSecurityComposition, PaperSecurityReduction은 목표 specification 이다.
easycrypt/security/ ImplementationSecurity Goals.ec	obl_*는 가정으로 숨기지 않을 구현 의무 목록이다. 파일 컴파일은 각 의무의 참을 뜻하지 않는다.
easycrypt/security/ SecurityAssumptions.ec	최종 theorem에 남길 수 있는 MLWE, BST-MSIS, ROM interface와 nonnegative bound의 well-formedness다. concrete reduction 연결은 별도다.

B 기호와 용어 사전

용어	이 문서에서의 뜻
명제(proposition)	참 또는 거짓을 판정할 수 있는 수학적 진술(mathematical statement); 이 문서에서는 정리(theorem)의 전제와 결론을 기술하는 문장도 포함함
정리(theorem)	전제(premise)와 허용된 추론 규칙(inference rule)으로 증명된 명제(proposition); EasyCrypt에서는 명명된 선언과 그 proof term을 함께 확인함
보조정리(lemma)	더 큰 정리(theorem)의 증명에 사용하는 중간 명제(intermediate proposition)
환(ring)	덧셈(addition)과 곱셈(multiplication)이 정의되고 환 공리(ring axioms)를 만족하는 대수적 구조(algebraic structure)
몫환(quotient ring)	환을 아이디얼(ideal)에 의한 동치관계로 나눈 환; 여기서는 주로 $R_q = \mathbb{Z}_q[X]/(X^n + 1)$
모듈(module)	환 위의 벡터공간(vector space)에 해당하는 구조; 계수의 스칼라가 체(field)가 아니라 환일 수 있음
사상(map)	한 대상의 원소를 다른 대상의 원소로 보내는 규칙; 문맥에 따라 함수(function)나 절차가 유도하는 의미론적 대응을 가리킴
동형사상(isomorphism)	역함수(inverse function)를 가지며 해당 구조를 보존하는 사상(map)
동치관계(equivalence relation)	반사성(reflexivity), 대칭성(symmetry), 추이성(transitivity)을 만족하는 관계(relation)
몫집합(quotient set)	동치관계(equivalence relation)의 동치류(equivalence class)를 원소로 하는 집합(set)
가환도식(commutative diagram)	같은 시작점과 끝점을 잇는 서로 다른 사상 합성이 같아지는 도식(diagram)
부분함수(partial function)	정의역(domain)의 모든 점이 아니라 지정된 부분에서만 값이 정의되는 함수(function)
불변식(invariant)	반복이나 상태전이(state transition) 전후에 계속 보존되는 명제(proposition)
전단사(bijection)	단사(injection)이면서 전사(surjection)인 함수(function)
mode 2	$n = 256, (k, \ell) = (2, 4), q = 64513, \eta = 1, \tau = 58$ 인 고정 parameter set
generated procedure	pinned Jasmin source에서 jasmin2ec로 추출된 실제 EasyCrypt procedure
focused extraction	특정 호출·table만 좁게 재추출한 target; full extraction과 procedure identity를 별도 증명·hash로 확인함
pure lemma	generated procedure 호출 없이 정수, list, 배열 연산만 다루는 정리(theorem)
leaf theorem	큰 절차의 한 작은 변환 또는 word 연산을 직접 여는 정리(theorem)
wrapper/adapter	이미 증명된 작은 표면과 actual generated top call을 연결하는 절차 또는 관계 정리(relational theorem)
raw/internal public/cryptolab caller	public context 포장 이전의 고정 raw branch와 내부 mode 2 entry 외부 pointer, length, copy, early-return을 포함한 실제 API 층
Hoare partial correctness	절차가 반환한다면 사후조건이 성립함; 종료성은 별도

용어	이 문서에서의 뜻
pRHL / equiv	두 프로그램의 관련 초기상태와 관련 종료상태를 연결하는 확률적 관계 Hoare logic 표현
exact trace	실제 결과·최종 메모리를 보존하면서 ghost observation을 노출하는 instrumented 절차와의 등가
frame	목표 밖 배열/메모리 구간이 실행 전후 동일하다는 정리(theorem)
정준성(canonicity)	encode/decode가 역이 되도록 선택한 word/계수(coefficient)의 대표자(representative) 조건
prefix	배열 전체가 아니라 $[0, n)$ 제한에 대한 점별 등식
region-local	전체 메모리 등식 대신 실제 read footprint 구간만 비교함
tail reached	Verify의 early-reject들을 지나 실제 tail/hash/challenge 경로에 도달했다는 ghost control 사실
bad	actual codec의 성공/실패 word; 보통 0은 성공, 1은 실패지만 각 정리(theorem)의 branch 조건을 확인해야 함
non-vacuity	전제 또는 성공 branch를 만족하는 concrete state/execution이 실제로 존재한다는 증거
zero-loss	명시된 국소 relational seam의 deterministic/statistical loss가 0임; 상위 δ 전체가 0이라는 뜻이 아님
-no-eco	EasyCrypt cache reuse 없이 authored target을 fresh compile하는 검증 옵션

C 재현과 독해 경로

C.1 이 안내서만 빌드하기

저장소 루트에서:

```
sh haetae-topdown-easycrypt/algebraist-guide/build.sh
```

성공하면 haetae-topdown-easycrypt/algebraist-guide/main.pdf가 생성된다. 스크립트는 source-of-truth 파일의 존재와 undefined reference/citation, 그리고 67개 snapshot과 proof-target manifest 수의 일치를 확인한다. PDF 빌드는 EasyCrypt 정리(theorem)를 재검증하지 않는다.

C.2 전체 증명 파이프라인

저장소 루트에서:

```
./haetae-topdown-easycrypt/scripts/verify-all.sh
```

파이프라인은 pinned source drift, focused extraction, generated certificate hash, authored target manifest, -no-eco compile, proof hole/axiom/debug scan, selected baseline, 기존 연구노트 LaTeX를 검사한다. Week 13 종료 기록상 authored target 수는 67이지만, 재현 시점의 실제 출력이 항상 우선한다.

검증된 명제(verified proposition) C.1 (문서 작성 시점의 aggregate 결과). WEEK13_REPORT.md와 최신 aggregate log에 기록된 전체 실행은 RESULT PASS, authored-targets=67, cache=-no-eco다. Week 10의 여섯 파일, Week 11의 일곱 encoder 파일, Week 12의 여덟 decoder 파일, Week 13의 두 core-composition 파일이 manifest completeness, proof-hole/axiom/debug scan

과 fresh compile 대상에 포함된다. 이 PASS는 명명된 타깃들의 컴파일과 감사(audit)를 뜻하며, 상위 **PARTIAL** 또는 **BLOCKED** 주장을 완료로 승격하지 않는다.

C.3 30분 독해 경로

1. PROJECT_SCOPE.md의 in/out-of-scope를 읽는다.
2. CLAIM_LEDGER.md에서 **KG-PREFIX-RETURN**, **OBL-DIRECT-OBSERVED-MU**, **OBL-RANS-ENCODE-REFINEMENT**, **OBL-RANS-DECODE-REFINEMENT**, **OBL-RANS-CORE-INVERSE**, **OBL-RANS-ACTUAL-SUCCESS-WITNESS** 여섯 행을 비교한다.
3. 본문의 sections 2, 5 and 6를 읽는다.
4. WEEK13_REPORT.md와 Mode2RansCoreActualInverse.ec에서 failure/success 사후조건(postconditions)과 아직 없는 full-HBZ wrapper 결론을 구분한다.

C.4 증명 참여 전 반나절 경로

1. RANS_BYTE_STACK_INVARIANT.md
Mode2RansByteStack.ec의 순수 trace를 대응시킨다.
2. RANS_ENCODER_INVARIANT.md
tail-aware 바깥/안쪽 루프 불변식(outer/inner-loop invariant)의 항을 읽는다.
3. Mode2RansEncoderActualTraceClosure.ec
실제 procedure를 여는 closure 정리(theorem)의 성공/실패 분기와 loop invariant를 대조한다.
4. Mode2RansEncoderOuterRefinement.ec
성공 suffix, offset/size, prefix frame을 펼친 따름정리(corollary)를 읽는다.
5. RANS_DECODER_INVARIANT.md
순수 state/cursor/segment 좌표(coordinates)와 actual outer/inner 불변식(invariants)을 대응시킨다.
6. Mode2RansDecoderActualTrace.ec
exact 입력 술어(input predicate), loop exit의 $x = 2^{23}$, 공개된 사후조건(postcondition)을 읽는다.
7. Mode2RansDecoderTopHoare.ec
실제 generated decoder를 직접 여는 정리(theorem)를 확인한다.
8. Mode2RansCoreCompositionBridge.ec에서 encoder/copy 결론을 decoder 입력 술어(input predicate)로 운송하는 bridge를 읽는다.
9. Mode2RansCoreActualInverse.ec에서 세 actual 호출의 성공 조건부 역함수(inverse function)를 읽고, production full-HBZ wrapper에 남은 size/frame 간극(gap)을 적는다.
10. 변경 전에 전체 verifier의 baseline을 보존한다.

마지막 판독 규칙

정리 이름(theorem name)이나 파일명이 아니라 명제(proposition)의 실제 procedure surface, 전제, 사후조건을 읽는다. “pure inverse”, “actual control”, “actual refinement”, “full parent”는 서로 다른 네 주장이다.

참고문헌

- [1] CryptoLab. HAETAE specification, 2026. Pinned local artifact CryptoLab_HAETAE.pdf, 46 pages, produced 2026-02-04; SHA-256 80d28b3af9af2a27dba0eb4746f9b5755e05cb9a86bb6596f45fda06c605f3f6.
- [2] CryptoLab. Pinned HAETAE jasmin implementation tree, 2026. Local source root haetae-ref-jasmin, used by focused jasmin2ec extraction.
- [3] HAETAE top-down EasyCrypt project. Claim ledger, 2026. Operational source of truth, local document haetae-topdown-easycrypt/CLAIM_LEDGER.md, Week 13 snapshot.
- [4] HAETAE top-down EasyCrypt project. Mode-2 actual rans core composition, 2026. Local document haetae-topdown-easycrypt/RANS_CORE_COMPOSITION.md.
- [5] HAETAE top-down EasyCrypt project. Mode-2 actual rans decoder invariant, 2026. Local document haetae-topdown-easycrypt/RANS_DECODER_INVARIANT.md.
- [6] HAETAE top-down EasyCrypt project. Mode-2 actual rans encoder invariant, 2026. Local document haetae-topdown-easycrypt/RANS_ENCODER_INVARIANT.md.
- [7] HAETAE top-down EasyCrypt project. Project scope and trust boundary, 2026. Pinned local document haetae-topdown-easycrypt/PROJECT_SCOPE.md.
- [8] HAETAE top-down EasyCrypt project. Theorem graph, 2026. Local dependency graph haetae-topdown-easycrypt/THEOREM_GRAPH.md.
- [9] HAETAE top-down EasyCrypt project. Week 10 report: Actual rans encoder inner loop and size bound, 2026. Local document haetae-topdown-easycrypt/WEEK10_REPORT.md.
- [10] HAETAE top-down EasyCrypt project. Week 11 report: Actual rans encoder trace closure, 2026. Local document haetae-topdown-easycrypt/WEEK11_REPORT.md.
- [11] HAETAE top-down EasyCrypt project. Week 12 report: Actual rans decoder semantic refinement, 2026. Local document haetae-topdown-easycrypt/WEEK12_REPORT.md.
- [12] HAETAE top-down EasyCrypt project. Week 13 report: Actual encoder/copy/decoder core inverse, 2026. Local document haetae-topdown-easycrypt/WEEK13_REPORT.md.
- [13] HAETAE top-down EasyCrypt project. Week 7 product/replay decision, 2026. Local document haetae-topdown-easycrypt/PRODUCT_REPLAY_DECISION.md.